คู่มือการใช้งาน

MOMO Joystick Controller

จอยบังคับไร้สายสำหรับงานหุ่นยนต์และระบบสมองกลฝังตัว

Wireless BLE Joystick Controller for ESP32 / ESP32-S3



รหัสสินค้า SKU-1001

ใช้งานร่วมกับบอร์ด ESP32 Breakout (SKU-1015) และ MOMO ESP32-S3 (SKU-1020)

บริษัท แมสมอร์ บิซ จำกัด

Massmore Biz Co., Ltd. · www.massmore.shop

รายการ	รายละเอียด
ชื่อเอกสาร	คู่มือการใช้งาน MOMO Joystick Controller
เลขที่เอกสาร	MM-SKU1001-UM-TH
ฉบับที่	1.1 (ตุลาคม 2569)
ไลบรารีที่อ้างอิง	Massmore_MomoJoy เวอร์ชัน 2.1.0
เครื่องมือพัฒนา	Arduino IDE 2.x และ ESP32 Arduino Core 3.x
จัดทำโดย	บริษัท แมสมอร์ บิซ จำกัด (Massmore Biz Co., Ltd.)

สารบัญ

1. ภาพรวมผลิตภัณฑ์ (Overview)
- 1.1 ความเป็นมาและวัตถุประสงค์
- 1.2 คุณสมบัติเด่น (Features)
- 1.3 องค์ประกอบของชุดและผลิตภัณฑ์ที่ใช้ร่วมกัน
- 1.4 ผังปุ่มและแกนควบคุม (Button Map)
- 1.5 ข้อปฏิบัติด้านความปลอดภัย

2. หลักการทำงานและข้อมูลจำเพาะ (Introduction & Specification)
- 2.1 หลักการทำงาน
- 2.2 ข้อมูลจำเพาะของจอยบังคับและการเชื่อมต่อ
- 2.3 ข้อมูลจำเพาะของซอฟต์แวร์
- 2.4 ข้อมูลจำเพาะของบอร์ดที่รองรับ
- 2.5 ข้อจำกัดของระบบ

3. การติดตั้งเครื่องมือพัฒนา (Arduino IDE)
- ขั้นตอนที่ 1 ติดตั้งโปรแกรม Arduino IDE 2.x
- ขั้นตอนที่ 2 ติดตั้งไดรเวอร์ USB-to-Serial
- ขั้นตอนที่ 3 เพิ่ม Board Manager URL ของ ESP32
- ขั้นตอนที่ 4 ติดตั้งแพ็กเกจบอร์ด ESP32 เวอร์ชัน 3.x
- ขั้นตอนที่ 5 ติดตั้งไลบรารี NimBLE-Arduino และ Massmore_MomoJoy
- ขั้นตอนที่ 6 ตั้งค่าบอร์ดในเมนู Tools
- ขั้นตอนที่ 7 ทดสอบการอัปโหลดโปรแกรม
- ขั้นตอนที่ 8 ตรวจสอบความพร้อมตามรายการตรวจสอบ

4. การเชื่อมต่อและอ่านค่าจอยบังคับ (User Guide)
- 4.1 การจับคู่จอยครั้งแรก (ตัวอย่าง 01_Pairing)
- 4.2 การอ่านค่าทั้งหมดจากจอย (ตัวอย่าง 02_ReadAllSerial)
- 4.3 การใช้งานจอยหลายตัวในพื้นที่เดียวกัน (ตัวอย่าง 03_PairByAddress)

5. การใช้งานกับบอร์ด ESP32 Breakout (SKU-1015)
- 5.1 ข้อมูลบอร์ด
- 5.2 อุปกรณ์ที่ต้องใช้
- 5.3 การต่อวงจร
- 5.4 การอัปโหลดโปรแกรมตัวอย่าง 04_Robot2WD_ESP32
- 5.5 วิธีการบังคับ
- 5.6 การปรับค่าการตอบสนอง
- 5.7 โครงสร้างโปรแกรมส่วนสำคัญ

6. การใช้งานกับบอร์ด MOMO ESP32-S3 (SKU-1020)
- 6.1 ข้อมูลบอร์ด
- 6.2 อุปกรณ์ที่ต้องใช้
- 6.3 การต่อวงจร
- 6.4 การอัปโหลดโปรแกรมตัวอย่าง 05_Robot4WD_ESP32S3
- 6.5 วิธีการบังคับ
- 6.6 การกำหนดชนิดล้อ
- 6.7 การปรับค่าการตอบสนอง

7. การทดสอบจอยบังคับ (Factory Test)
- 7.1 วัตถุประสงค์
- 7.2 ขั้นตอนการทดสอบ
- 7.3 รายงานผลการทดสอบ
- 7.4 เฟิร์มแวร์สำเร็จรูป

8. การแก้ไขปัญหา (Troubleshooting)
- 8.1 ปัญหาการคอมไพล์และอัปโหลด
- 8.2 ปัญหาการเชื่อมต่อจอย
- 8.3 ปัญหาระหว่างการใช้งานหุ่นยนต์
- 8.4 การปรับเทียบปุ่มสำหรับจอยรุ่นอื่น

9. ข้อแนะนำสำหรับการใช้งาน (Pro Tips)

10. แหล่งข้อมูลอ้างอิง (Resources)
- 10.1 คลังข้อมูล GitHub
- 10.2 ผลิตภัณฑ์ที่เกี่ยวข้อง
- 10.3 มาตรฐานที่อ้างอิง
- 10.4 เอกสารข้อมูลวงจรรวม (Datasheet)
- 10.5 เว็บไซต์อ้างอิง
- 10.6 สำหรับผู้ใช้งาน PlatformIO

ภาคผนวก ก สรุปคำสั่งของไลบรารี

ภาคผนวก ข แบบบันทึกผลการทดสอบจอยบังคับ

1. ภาพรวมผลิตภัณฑ์ (Overview)

1.1 ความเป็นมาและวัตถุประสงค์

MOMO Joystick Controller เป็นจอยบังคับไร้สายที่สื่อสารด้วยมาตรฐาน **Bluetooth Low Energy (BLE)** ออกแบบให้เชื่อมต่อกับบอร์ดไมโครคอนโทรลเลอร์ตระกูล **ESP32 และ ESP32-S3** ได้โดยตรง โดยไม่ต้องใช้โทรศัพท์มือถือ คอมพิวเตอร์ หรืออุปกรณ์รับสัญญาณ (ดองเกิล) เพิ่มเติม เหมาะสำหรับใช้เป็นอุปกรณ์ควบคุมหุ่นยนต์ในการเรียนการสอน การแข่งขัน และงานต้นแบบ

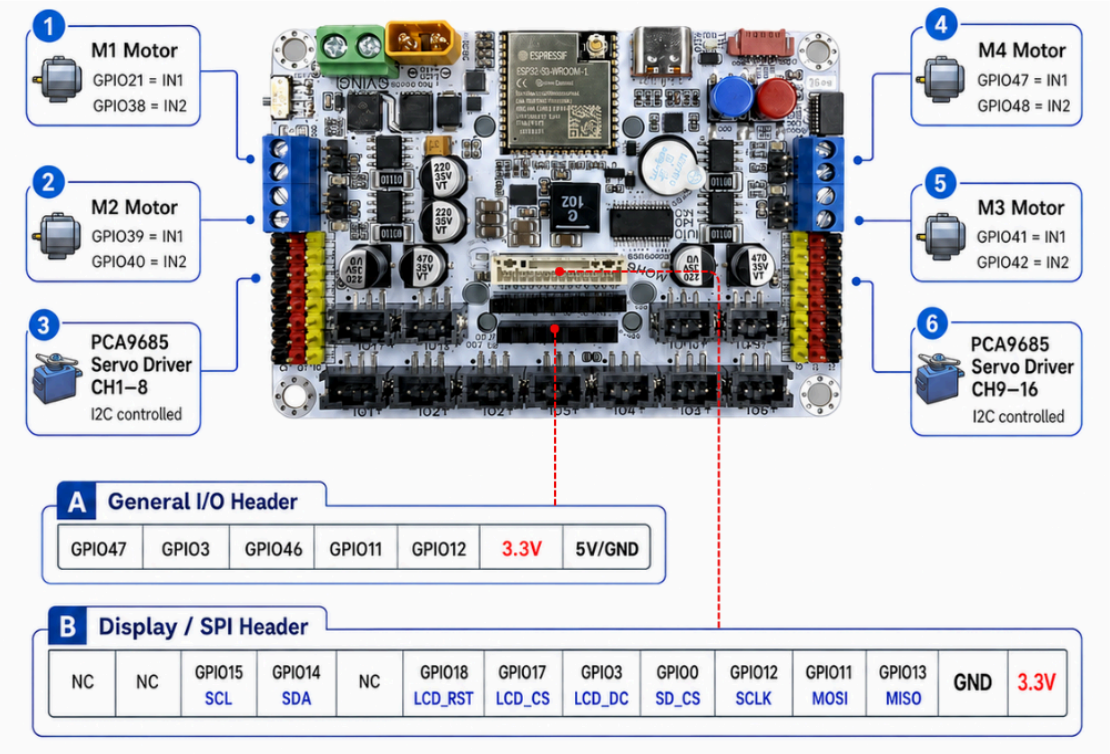
การใช้งานอาศัยไลบรารี **Massmore_MomoJoy** ซึ่งทำให้บอร์ด ESP32 ทำหน้าที่เป็น **BLE HID Host** คือเป็นฝ่ายค้นหา เชื่อมต่อ และรับข้อมูลจากจอยบังคับ ข้อมูลที่ได้ ได้แก่ ค่าก้านอนาล็อก ค่าไกอนาล็อก สถานะปุ่มกด และระดับแบตเตอรี่ ซึ่งอยู่ในรูปตัวเลขที่นำไปควบคุมมอเตอร์หรือเซอร์โวได้ทันที

คู่มือฉบับนี้มีวัตถุประสงค์เพื่อ

1. อธิบายหลักการทำงานและข้อมูลจำเพาะของจอยบังคับและไลบรารี
2. แนะนำขั้นตอนการติดตั้งเครื่องมือพัฒนาโดยละเอียด ตั้งแต่เริ่มต้นจนพร้อมใช้งาน
3. อธิบายการใช้งานร่วมกับบอร์ด **ESP32 Breakout (SKU-1015)** และ **MOMO ESP32-S3 (SKU-1020)** พร้อมโปรแกรมตัวอย่าง
4. รวบรวมแนวทางการแก้ไขปัญหาและแหล่งข้อมูลอ้างอิงสำหรับการศึกษาเพิ่มเติม



ภาพที่ 1 ภาพผลิตภัณฑ์ MOMO Joystick Controller มุมมองด้านหน้า



ภาพที่ 2 แผนผังการทำงานของระบบ: จอยบังคับ → (สัญญาณ BLE) → บอร์ด ESP32 / ESP32-S3 → วงจรขับมอเตอร์ → มอเตอร์และเซอร์โวของหุ่นยนต์

1.2 คุณสมบัติเด่น (Features)

- **เชื่อมต่อกับ ESP32 ได้โดยตรง** บอร์ดทำหน้าที่เป็น BLE HID Host เอง ไม่ต้องใช้อุปกรณ์ตัวกลาง
- **ใช้มาตรฐาน Bluetooth Low Energy** ใช้พลังงานต่ำ จับคู่ครั้งเดียวแล้วบอร์ดจดจำไว้ เมื่อเปิดเครื่องครั้งถัดไปจะเชื่อมต่อให้อัตโนมัติ
- **ก้านอนาล็อก 2 ชุด** ให้ค่าละเอียดตั้งแต่ –512 ถึง 511 พร้อมระบบ Dead Zone แบบรัศมีที่ปรับค่าได้
- **ไกอนาล็อก L2 / R2** ให้ค่าตั้งแต่ 0 ถึง 1023 จึงใช้เป็นคันเร่งแบบปรับระดับได้
- **ปุ่มกดสูงสุด 18 ปุ่ม** และ **D-Pad** ที่แปลงเป็นทิศทางขึ้น ลง ซ้าย ขวา ให้พร้อมใช้งาน
- **อ่านระดับแบตเตอรี่ของจอยได้** ในช่วง 0–100% ผ่าน Battery Service มาตรฐาน
- **ระบุจอยด้วย MAC Address ได้** ป้องกันการเชื่อมต่อสลับเครื่อง เมื่อใช้งานจอยหลายตัวในห้องเดียวกัน
- **แปลความหมายข้อมูล HID ขณะทำงาน (Runtime HID Report Map Parsing)** จึงรองรับจอย BLE HID ทั่วไปได้ ไม่ผูกกับจอยรุ่นใดรุ่นหนึ่ง
- **มีไฟล์ติดตั้งพร้อมใช้** ในรูปแบบ .zip สำหรับ Arduino IDE และโปรเจกต์สำหรับ PlatformIO โดยรวมไลบรารี NimBLE-Arduino ไว้ในชุดแล้ว
- **โปรแกรมตัวอย่าง 6 ชุด** เรียงจากระดับพื้นฐานถึงระดับประยุกต์ ครอบคลุมทั้งบอร์ด ESP32 และ ESP32-S3

1.3 องค์ประกอบของชุดและผลิตภัณฑ์ที่ใช้ร่วมกัน

รายการ	รหัสสินค้า	บทบาทในระบบ
MOMO Joystick Controller	SKU-1001	อุปกรณ์ควบคุม ส่งค่าปุ่มและก้านผ่าน BLE
บอร์ดขยาย ESP32 38PIN Breakout	SKU-1015	บอร์ดควบคุมหุ่นยนต์ 2 ล้อ ใช้ร่วมกับโมดูลขับมอเตอร์ภายนอก

รายการ	รหัสสินค้า	บทบาทในระบบ
บอร์ดพัฒนาหุ่นยนต์ MOMO ESP32-S3	SKU-1020	บอร์ดควบคุมหุ่นยนต์ 4 ล้อ มีวงจรขับมอเตอร์ 4 ช่องบนบอร์ด
ไลบรารี Massmore_MomoJoy	—	ซอฟต์แวร์สำหรับเชื่อมต่อและอ่านค่าจอยบนบอร์ด ESP32 / ESP32-S3

1.4 ผังปุ่มและแกนควบคุม (Button Map)

จอยบังคับสื่อสารแบบไร้สาย จึงไม่มีขาสัญญาณสำหรับต่อสาย ข้อมูลที่ใช้แทนผังขา (Pinout) คือผังปุ่มและช่วงค่าที่โปรแกรมอ่านได้ ดังรายละเอียดต่อไปนี้



ภาพที่ 3 ผังปุ่มด้านหน้าของจอยบังคับ ระบุ A, B, X, Y, D-Pad, ก้านซ้าย (L2), ก้านขวา (R2), SELECT, START, HOME และไฟแสดงสถานะ

1. อนาล็อกซ้าย (Left Joystick / L2) ใช้ควบคุมการเคลื่อนที่ของตัวละคร เช่น เดิน วิ่ง หรือบังคับทิศทาง สามารถกดอนาล็อกลงเพื่อใช้งานปุ่ม L2 ได้

2. ปุ่ม View (View Button) ใช้เปิดหน้าต่างข้อมูล แผนที่ หรือเมนูย่อยภายในเกม ขึ้นอยู่กับเกมที่เล่น

3. ปุ่ม Home (Home Button) ใช้เรียกเมนูหลักของระบบหรือจอยเกม และอาจใช้สำหรับเปิดเครื่องหรือเชื่อมต่ออุปกรณ์ตามโหมดการทำงาน

4. ปุ่ม Menu (Menu Button) ใช้เปิดเมนูตัวเลือกภายในเกม เช่น หยุดเกมชั่วคราว (Pause) ตั้งค่าเกม หรือเปิดเมนูคำสั่ง

5. ปุ่ม O (O Button) เป็นปุ่มฟังก์ชันพิเศษของจอย การทำงานขึ้นอยู่กับโหมดการเชื่อมต่อและการตั้งค่าของจอย

6. ปุ่ม T (Turbo Button) ใช้สำหรับฟังก์ชัน Turbo ช่วยให้ปุ่มที่รองรับสามารถส่งคำสั่งกดซ้ำอย่างต่อเนื่อง โดยไม่ต้องกดปุ่มหลายครั้งด้วยตนเอง

7. แป้นทิศทาง (D-Pad) ใช้ควบคุมทิศทาง 4 ทาง ได้แก่ บน ล่าง ซ้าย และขวา เหมาะสำหรับเลือกเมนู เปลี่ยนอุปกรณ์ หรือควบคุมตัวละครภายในเกม

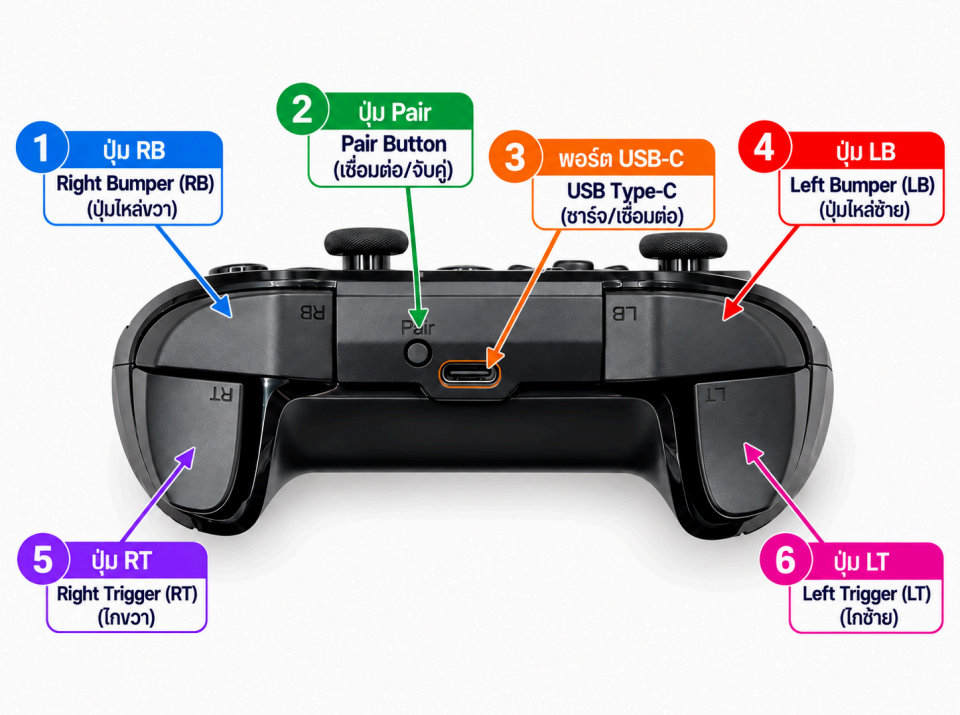
8. ปุ่ม Y (Y Button) เป็นปุ่มคำสั่งหลักของเกม ใช้สำหรับกระโดด เปลี่ยนอาวุธ โต้ตอบ หรือคำสั่งอื่น ๆ ตามการตั้งค่าของแต่ละเกม

9. ปุ่ม X (X Button) เป็นปุ่มคำสั่งหลักของเกม ใช้สำหรับโจมตี หยิบสิ่งของ หรือทำกิจกรรมต่าง ๆ ภายในเกม

10. ปุ่ม B (B Button) โดยทั่วไปใช้สำหรับยกเลิกคำสั่ง ย้อนกลับ ออกจากเมนู หรือใช้เป็นปุ่มควบคุมภายในเกม

11. ปุ่ม A (A Button) โดยทั่วไปใช้สำหรับยืนยันคำสั่ง เลือกเมนู โต้ตอบกับสิ่งของ หรือใช้งานตามที่เกมกำหนด

12. อนาล็อกขวา (Right Joystick / R2) ใช้ควบคุมมุมกล้อง การมองซ้าย-ขวา หรือการเล็งเป้าหมาย สามารถกดอนาล็อกลงเพื่อใช้งานปุ่ม R2 ได้



ภาพที่ 4 ผังปุ่มด้านบนของจอยบังคับ ระบุ L1, R1, L2, R2 และตำแหน่งสวิตช์เลือกโหมด

1.4.1 แกนอนาล็อก (Analog Axes)

ส่วนควบคุม	คำสั่งที่ใช้อ่านค่า	ช่วงค่า	หมายเหตุ
ก้านซ้าย แกน X	MomoJoy.lx()	−512 ถึง 511	0 = ตำแหน่งกลาง, ค่าบวก = ด้านขวา
ก้านซ้าย แกน Y	MomoJoy.ly()	−512 ถึง 511	**ดันขึ้นได้ค่าติดลบ**
ก้านขวา แกน X	MomoJoy.rx()	−512 ถึง 511	0 = ตำแหน่งกลาง, ค่าบวก = ด้านขวา
ก้านขวา แกน Y	MomoJoy.ry()	−512 ถึง 511	ดันขึ้นได้ค่าติดลบ

ส่วนควบคุม	คำสั่งที่ใช้อ่านค่า	ช่วงค่า	หมายเหตุ
ไกซ้าย L2	MomoJoy.l2()	0 ถึง 1023	ค่าเพิ่มขึ้นตามระยะที่กด
ไกขวา R2	MomoJoy.r2()	0 ถึง 1023	ค่าเพิ่มขึ้นตามระยะที่กด

ทุกแกนมีคำสั่งรูปแบบทศนิยม (float) สำหรับการคำนวณ ได้แก่ lxf() lyf() rxf() ryf() ให้ค่า –1.0 ถึง 1.0 และ l2f() r2f() ให้ค่า 0.0 ถึง 1.0

> **หมายเหตุ** แกน Y ให้ค่าติดลบเมื่อดันก้านขึ้นตามมาตรฐาน HID หากต้องการใช้เป็นคำสั่งเดินหน้า ต้องกลับเครื่องหมายก่อน เช่น int speed = -MomoJoy.ly(); หากไม่กลับเครื่องหมาย หุ่นยนต์จะเคลื่อนที่ถอยหลังเมื่อดันก้านไปข้างหน้า

1.4.2 ปุ่มกด (Buttons)

ค่าคงที่ในโปรแกรม	ปุ่มบนจอย	ค่าคงที่ในโปรแกรม	ปุ่มบนจอย
MOMO_BTN_A	A	MOMO_BTN_L3	กดก้านซ้ายลง
MOMO_BTN_B	B	MOMO_BTN_R3	กดก้านขวาลง
MOMO_BTN_X	X	MOMO_BTN_SELECT	SELECT
MOMO_BTN_Y	Y	MOMO_BTN_START	START
MOMO_BTN_L1	ปุ่มบ่าซ้าย	MOMO_BTN_HOME	HOME
MOMO_BTN_R1	ปุ่มบ่าขวา	MOMO_BTN_M1, MOMO_BTN_M2	ปุ่มเสริมด้านหลัง (หากมี)
MOMO_BTN_L2	ไกซ้าย (สถานะกด/ไม่กด)	MOMO_BTN_C, MOMO_BTN_Z	ปุ่มเสริม (หากมี)
MOMO_BTN_R2	ไกขวา (สถานะกด/ไม่กด)	MOMO_BTN_CAPTURE	ปุ่ม CAPTURE (หากมี)

สถานะกดของ L2 / R2 จะเป็นจริงเมื่อค่าอนาล็อกเกินค่าเกณฑ์ (ค่าเริ่มต้น 64) ซึ่งปรับได้ด้วย setTriggerThreshold()

1.4.3 ปุ่มทิศทาง (D-Pad)

ค่าคงที่ในโปรแกรม	ทิศทาง	คำสั่งอ่านค่าโดยตรง
MOMO_DPAD_UP	ขึ้น	MomoJoy.dpadUp()
MOMO_DPAD_DOWN	ลง	MomoJoy.dpadDown()
MOMO_DPAD_LEFT	ซ้าย	MomoJoy.dpadLeft()
MOMO_DPAD_RIGHT	ขวา	MomoJoy.dpadRight()

1.4.4 รูปแบบการอ่านสถานะปุ่ม

คำสั่ง	ให้ผลเป็นจริงเมื่อ	ตัวอย่างการนำไปใช้
pressed(ปุ่ม)	ตลอดช่วงเวลาที่กดค้าง	เร่งความเร็ว เปิดเสียงสัญญาณ
justPressed(ปุ่ม)	ครั้งเดียว ณ จังหวะที่เริ่มกด	สลับโหมด เปิด/ปิดฟังก์ชัน
justReleased(ปุ่ม)	ครั้งเดียว ณ จังหวะที่ปล่อย	หยุดการทำงานเมื่อปล่อยปุ่ม

คำสั่ง	ให้ผลเป็นจริงเมื่อ	ตัวอย่างการนำไปใช้
longPressed(ปุ่ม, มิลลิวินาที)	กดค้างครบระยะเวลาที่กำหนด	ล้างข้อมูลการจับคู่ รีเซ็ตค่า

หมายเหตุ คำสั่ง pressed() รองรับการระบุหลายปุ่มพร้อมกันโดยคั่นด้วยเครื่องหมาย | และจะให้ผลเป็นจริงเมื่อกดปุ่มใดปุ่มหนึ่ง เช่น pressed(MOMO_BTN_L1 | MOMO_BTN_R1) หากต้องการเงื่อนไขกดพร้อมกันทั้งสองปุ่ม ให้เขียนเป็น pressed(MOMO_BTN_L1) && pressed(MOMO_BTN_R1)

1.4.5 ข้อมูลสถานะการเชื่อมต่อ

คำสั่ง	ค่าที่ได้	การนำไปใช้
MomoJoy.isConnected()	true / false	ตรวจสอบก่อนสั่งงานมอเตอร์ทุกครั้ง
MomoJoy.battery()	0–100 (0xFF = ไม่ทราบค่า)	แจ้งเตือนเมื่อแบตเตอรี่ของจอยใกล้หมด
MomoJoy.rssi()	ความแรงสัญญาณ หน่วย dBm	ค่าใกล้ 0 หมายถึงสัญญาณแรง
MomoJoy.reportRateHz()	จำนวนครั้งต่อวินาที	ตรวจสอบคุณภาพการเชื่อมต่อ
MomoJoy.msSinceReport()	มิลลิวินาที	ใช้ทำระบบหยุดฉุกเฉินเมื่อสัญญาณขาดหาย
MomoJoy.peerName() / peerAddress()	ข้อความ	ชื่อและ MAC Address ของจอยที่เชื่อมต่ออยู่

1.5 ข้อปฏิบัติด้านความปลอดภัย

ผู้ใช้งานและผู้สอนควรปฏิบัติตามข้อกำหนดต่อไปนี้ทุกครั้งที่ทดลองกับหุ่นยนต์ที่มีมอเตอร์

1. ยกล้อหุ่นยนต์ให้พ้นพื้นทุกครั้งที่อัปโหลดโปรแกรมหรือทดสอบครั้งแรก
2. ตรวจสอบขั้วแบตเตอรี่และแรงดันไฟเลี้ยงให้อยู่ในช่วงที่บอร์ดกำหนดก่อนต่อวงจร
3. ปิดสวิตช์ไฟเลี้ยงก่อนต่อหรือถอดสายมอเตอร์ทุกครั้ง
4. ทดสอบปุ่มหยุดฉุกเฉิน (ปุ่ม **B** ในโปรแกรมตัวอย่าง) ก่อนวางหุ่นยนต์ลงพื้น
5. ไม่ปล่อยให้หุ่นยนต์ทำงานโดยไม่มีผู้ควบคุม และจัดพื้นที่ทดสอบให้ห่างจากขอบโต๊ะ

ข้อควรระวัง โปรแกรมตัวอย่างทุกชุดกำหนดขาควบคุมมอเตอร์ให้เป็นสถานะ LOW เป็นคำสั่งแรกในฟังก์ชัน setup() ผ่านฟังก์ชัน parkMotorPins() เพื่อป้องกันมอเตอร์หมุนเองขณะบอร์ดเริ่มทำงาน เมื่อเขียนโปรแกรมใหม่สำหรับบอร์ดหุ่นยนต์ ผู้ใช้งานควรคงขั้นตอนนี้ไว้เสมอ

2. หลักการทำงานและข้อมูลจำเพาะ (Introduction & Specification)

2.1 หลักการทำงาน

จอยบังคับไร้สายทั่วไปมักใช้อุปกรณ์รับสัญญาณ 2.4 GHz เฉพาะรุ่น ซึ่งต้องเสียบเข้าพอร์ต USB และไม่สามารถใช้ร่วมกับบอร์ดไมโครคอนโทรลเลอร์ได้โดยตรง จอยบังคับรุ่นนี้ใช้มาตรฐาน **HID over GATT Profile (HOGP)** ซึ่งเป็นโปรไฟล์ของ Bluetooth Low Energy ที่กำหนดโดย Bluetooth SIG สำหรับอุปกรณ์รับคำสั่งจากผู้ใช้ เช่น แป้นพิมพ์ เมาส์ และจอยเกม เนื่องจาก ESP32 มีวงจร Bluetooth อยู่ภายในชิป ไลบรารีจึงทำให้บอร์ดเป็นฝ่ายเชื่อมต่อเข้าหาจอยได้โดยตรง

ลำดับการทำงานเมื่อบอร์ดเริ่มทำงาน

1. บอร์ดสแกนหาอุปกรณ์ที่ประกาศ HID Service (0x1812) หรือ Appearance = Gamepad (0x03C4)
2. พบจอย -> เชื่อมต่อและจับคู่ (Just Works + LE Secure Connections) แล้วบันทึกข้อมูลลงหน่วยความจำ NVS
3. ตั้งค่า Protocol Mode เป็น Report (Characteristic 0x2A4E)
4. อ่าน Report Map แล้วแปลเป็นตารางข้อมูล (Characteristic 0x2A4B)
5. ลงทะเบียนรับข้อมูลจาก Input Report (Characteristic 0x2A4D, Descriptor 0x2908)
6. จอยส่งข้อมูลแบบ Notify -> คิวของ FreeRTOS -> ถอดรหัส -> ได้ค่าพร้อมใช้งาน

ขั้นตอนที่สำคัญคือการ **อ่าน Report Map** ซึ่งจอยจะระบุเองว่าข้อมูลแต่ละไบต์และแต่ละบิตหมายถึงแกนหรือปุ่มใด ไลบรารีจึงไม่ต้องกำหนดตำแหน่งข้อมูลไว้ตายตัว และสามารถใช้กับจอย BLE HID รุ่นอื่นได้ ส่วนที่อาจแตกต่างกันระหว่างรุ่นมีเพียงหมายเลขปุ่ม ซึ่งปรับได้ตามวิธีในหัวข้อ 8.4



ภาพที่ 5 ตำแหน่งสวิตช์เลือกโหมด ตำแหน่ง **D**

หมายเหตุ ต้องเลื่อนสวิตช์โหมดของจอยไปที่ตำแหน่ง **D** ทุกครั้งที่ใช้งานกับบอร์ด ESP32 เนื่องจากตำแหน่งนี้เป็นโหมด **BLE HID** ซึ่งเป็นโหมดเดียวที่ ESP32 สื่อสารด้วยได้ โหมดอื่นของจอยใช้ Bluetooth Classic หรือการสื่อสารเฉพาะ ซึ่ง ESP32-S3 ไม่รองรับ

2.2 ข้อมูลจำเพาะของจอยบังคับและการเชื่อมต่อ

หัวข้อ	รายละเอียด
มาตรฐานการเชื่อมต่อ	Bluetooth Low Energy, HID over GATT (HID Service 0x1812)
โหมดที่ใช้งาน	สวิตช์โหมดตำแหน่ง **D** (BLE HID)
การจับคู่	Just Works ร่วมกับ LE Secure Connections บันทึกข้อมูลการจับคู่ลง NVS
ก้านอนาล็อก	2 ชุด ชุดละ 2 แกน ค่า –512 ถึง 511 พร้อม Dead Zone แบบรัศมี (ค่าเริ่มต้น 24)
ไกอนาล็อก	L2 / R2 ค่า 0 ถึง 1023
D-Pad	Hat Switch 8 ทิศทาง แปลงเป็นบิตทิศทาง 4 บิต
ปุ่มกด	สูงสุด 18 ปุ่ม
ระดับแบตเตอรี่	อ่านผ่าน Battery Service (0x180F) ค่า 0–100%
อัตราการส่งข้อมูล	ประมาณ 60–125 ครั้งต่อวินาที (ตรวจสอบได้ด้วย reportRateHz())
จำนวนจอยต่อบอร์ด	1 ตัวต่อบอร์ด 1 ใบ

2.3 ข้อมูลจำเพาะของซอฟต์แวร์

หัวข้อ	รายละเอียด
ชื่อไลบรารี	Massmore_MomoJoy เวอร์ชัน 2.1.0
ไมโครคอนโทรลเลอร์ที่รองรับ	ESP32 และ ESP32-S3
Arduino Core	**ESP32 Arduino Core เวอร์ชัน 3.x** (จำเป็น)
PlatformIO	pioarduino platform-espressif32 เวอร์ชัน 55.03.311
ไลบรารีที่ใช้ร่วม	NimBLE-Arduino 2.x (รวมไว้ในชุดแล้ว)
ขนาดโปรแกรมโดยประมาณ	Flash ประมาณ 590 KB, RAM ประมาณ 37 KB (วัดจากตัวอย่าง 06_FactoryTest)
สัญญาอนุญาต	MIT License

ข้อควรระวัง ไลบรารีต้องใช้กับ ESP32 Arduino Core เวอร์ชัน **3.x** เท่านั้น เนื่องจากโปรแกรมตัวอย่างใช้คำสั่งควบคุม PWM ชุดใหม่ (ledcAttach() และ ledcWrite()) หากใช้ Core เวอร์ชัน 2.x จะไม่สามารถคอมไพล์ได้

2.4 ข้อมูลจำเพาะของบอร์ดที่รองรับ

หัวข้อ	ESP32 Breakout (SKU-1015)	MOMO ESP32-S3 (SKU-1020)
ไมโครคอนโทรลเลอร์	โมดูล ESP32 38PIN (ESP32-WROOM-32)	ESP32-S3-WROOM-1U-N16R8
หน่วยความจำ	Flash 4 MB	Flash 16 MB, PSRAM 8 MB (OPI)
วงจรแปลง USB เป็น Serial	อยู่บนโมดูล ESP32 (ตามรุ่นของโมดูล)	CH343P
แรงดันไฟเลี้ยงขาเข้า	6–12 V (มี DC-DC แปลงเป็น 5 V / 3 A)	6–16 V ผ่าน Terminal Block หรือ XT30
วงจรขับมอเตอร์	ไม่มีบนบอร์ด ใช้โมดูลขับมอเตอร์ภายนอก (TB6612FNG หรือเทียบเท่า)	TB67H451AFNG จำนวน 4 ช่อง
ช่องมอเตอร์ในโปรแกรมตัวอย่าง	2 ช่อง (M1, M2)	4 ช่อง (M1–M4) พร้อม Encoder
ช่องเซอร์โว	3 ช่อง (GPIO 19, 32, 33)	ผ่านวงจร PCA9685PW (I2C SDA 14 / SCL 15)
อุปกรณ์บนบอร์ดที่ตัวอย่างใช้	ตัวต้านทานปรับค่า VR (GPIO 34), สวิตช์ SW (GPIO 36)	Buzzer (GPIO 45)
วงจรจ่ายไฟ 3.3 V	อยู่บนโมดูล ESP32	TPS62162
โปรแกรมตัวอย่างที่ใช้	04_Robot2WD_ESP32	05_Robot4WD_ESP32S3

2.5 ข้อจำกัดของระบบ

1. รองรับเฉพาะจอยที่สื่อสารแบบ **BLE HID** เท่านั้น จอยที่ใช้ Bluetooth Classic หรือจอยที่ต้องใช้อุปกรณ์รับสัญญาณ 2.4 GHz ไม่สามารถใช้งานร่วมกับชุดนี้ได้
2. บอร์ด 1 ใบเชื่อมต่อกับจอยได้ครั้งละ 1 ตัว
3. ระยะการสื่อสารขึ้นอยู่กับสภาพแวดล้อม สัญญาณ Wi-Fi และอุปกรณ์ที่ใช้คลื่นความถี่ 2.4 GHz บริเวณใกล้เคียง อาจทำให้การเชื่อมต่อไม่เสถียร

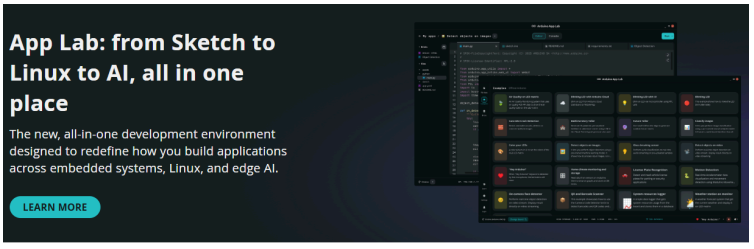
3. การติดตั้งเครื่องมือพัฒนา (Arduino IDE)

บทนี้อธิบายการเตรียมคอมพิวเตอร์ให้พร้อมเขียนโปรแกรมและอัปโหลดลงบอร์ดทั้งสองรุ่น ใช้เวลาประมาณ 20–30 นาที ขึ้นอยู่กับความเร็วอินเทอร์เน็ต

ลำดับขั้นตอนการดำเนินงาน (Road Map)
ขั้นตอนที่ 1ติดตั้งโปรแกรม Arduino IDE 2.x
ขั้นตอนที่ 2ติดตั้งไดรเวอร์ USB-to-Serial ของบอร์ด
ขั้นตอนที่ 3เพิ่ม Board Manager URL ของ ESP32
ขั้นตอนที่ 4ติดตั้งแพ็กเกจบอร์ด ESP32 เวอร์ชัน 3.x
ขั้นตอนที่ 5ติดตั้งไลบรารี NimBLE-Arduino และ Massmore_MomoJoy
ขั้นตอนที่ 6ตั้งค่าบอร์ดในเมนู Tools
ขั้นตอนที่ 7ทดสอบการอัปโหลดโปรแกรม
ขั้นตอนที่ 8ตรวจสอบความพร้อมตามรายการตรวจสอบ

ขั้นตอนที่ 1 ติดตั้งโปรแกรม Arduino IDE 2.x

ดาวน์โหลดโปรแกรมจาก https://www.arduino.cc/en/software โดยเลือกให้ตรงกับระบบปฏิบัติการ





ระบบปฏิบัติการ Windows

1. ดาวน์โหลดไฟล์ติดตั้งแบบ **Windows Win 10 and newer, 64 bits**
2. ดับเบิลคลิกไฟล์ .exe แล้วกด **I Agree** และ **Next** ตามขั้นตอน
3. หากโปรแกรมถามเรื่องการติดตั้งไดรเวอร์ USB ให้กด **Install** ทุกรายการ

ระบบปฏิบัติการ macOS

1. ดาวน์โหลดไฟล์ .dmg ให้ตรงกับชนิดของเครื่อง (**Apple Silicon** สำหรับชิป M1 ขึ้นไป หรือ **Intel**)
2. เปิดไฟล์ .dmg แล้วลากไอคอน Arduino IDE ไปวางในโฟลเดอร์ **Applications**
3. เปิดใช้งานครั้งแรก หากระบบแจ้งเตือนด้านความปลอดภัย ให้กด **Open**

ระบบปฏิบัติการ Linux

1. ดาวน์โหลดไฟล์ **AppImage 64 bits (X86-64)**
2. กำหนดสิทธิ์ให้เรียกใช้งานได้ด้วยคำสั่ง chmod +x arduino-ide_*.AppImage
3. เพิ่มสิทธิ์การใช้งานพอร์ตอนุกรมให้บัญชีผู้ใช้ แล้วออกจากระบบและเข้าสู่ระบบใหม่หนึ่งครั้ง

```
sudo usermod -a -G dialout $USER
```

> **หมายเหตุ** แนะนำให้ใช้ Arduino IDE เวอร์ชัน 2.x เนื่องจากรองรับ ESP32 Arduino Core 3.x ได้สมบูรณ์ และมีระบบจัดการบอร์ดและไลบรารีที่ใช้งานสะดวกกว่าเวอร์ชัน 1.8

ขั้นตอนที่ 2 ติดตั้งไดรเวอร์ USB-to-Serial

คอมพิวเตอร์สื่อสารกับบอร์ดผ่านวงจรแปลง USB เป็น Serial ซึ่งแต่ละบอร์ดใช้ชิปต่างกัน ให้ติดตั้งไดรเวอร์ให้ตรงกับบอร์ดที่ใช้

บอร์ด	ชิป USB-to-Serial	ชื่อพอร์ตที่ปรากฏโดยทั่วไป
MOMO ESP32-S3 (SKU-1020)	CH343P	Windows: COMx · macOS: /dev/cu.usbmodemxxxx หรือ /dev/cu.wchusbserialxxxx · Linux: /dev/ttyACM0
ESP32 Breakout (SKU-1015)	ตามรุ่นของโมดูล ESP32 (โดยทั่วไปเป็น CP2102 หรือ CH340)	Windows: COMx · macOS: /dev/cu.usbserial-xxxx หรือ /dev/cu.SLAB_USBtoUART · Linux: /dev/ttyUSB0

การติดตั้งไดรเวอร์ CH343P สำหรับบอร์ด MOMO ESP32-S3

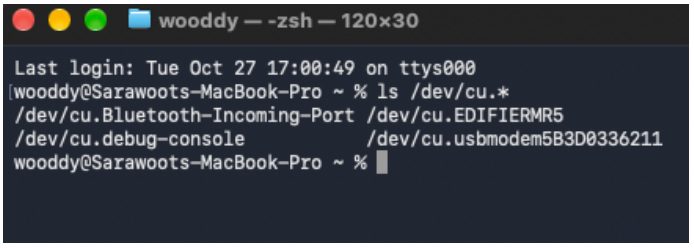
1. **Windows 10/11** โดยทั่วไประบบจะติดตั้งไดรเวอร์ให้อัตโนมัติผ่าน Windows Update หากไม่พบพอร์ต ให้ดาวน์โหลดไดรเวอร์ **CH343SER** จากเว็บไซต์ผู้ผลิต https://www.wch-ic.com/products/CH343.html แล้วติดตั้ง
2. **macOS 11 ขึ้นไป** ระบบรองรับชิปนี้ในตัว พอร์ตจะปรากฏเป็น /dev/cu.usbmodemxxxx หากต้องการใช้ไดรเวอร์ของผู้ผลิต ให้ดาวน์โหลดจากเว็บไซต์เดียวกัน แล้วอนุญาตส่วนขยายระบบที่ **System Settings → Privacy & Security**
3. **Linux** เคอร์เนลรองรับผ่านไดรเวอร์ cdc_acm ในตัว ไม่ต้องติดตั้งเพิ่ม

การติดตั้งไดรเวอร์สำหรับบอร์ด ESP32 Breakout

ตรวจสอบชื่อชิปที่พิมพ์อยู่บนโมดูล ESP32 แล้วติดตั้งไดรเวอร์ตามรุ่น ได้แก่ CP2102 จาก https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers หรือ CH340 จาก https://www.wch-ic.com/downloads/CH341SER_EXE.html

การตรวจสอบว่าไดรเวอร์ทำงาน

1. เสียบสาย USB ระหว่างคอมพิวเตอร์กับบอร์ด
2. Windows: เปิด **Device Manager** แล้วตรวจสอบหัวข้อ **Ports (COM & LPT)** ว่ามีพอร์ตใหม่ปรากฏขึ้น
3. macOS / Linux: เปิด Terminal แล้วพิมพ์คำสั่ง ls /dev/cu.* (macOS) หรือ ls /dev/tty{USB,ACM}* (Linux)



/dev/cu.usbmodem5B3D0336211

ภาพที่ 6 หน้าต่าง Device Manager แสดงพอร์ต COM ของชิป CH343P หลังติดตั้งไดรเวอร์สำเร็จ

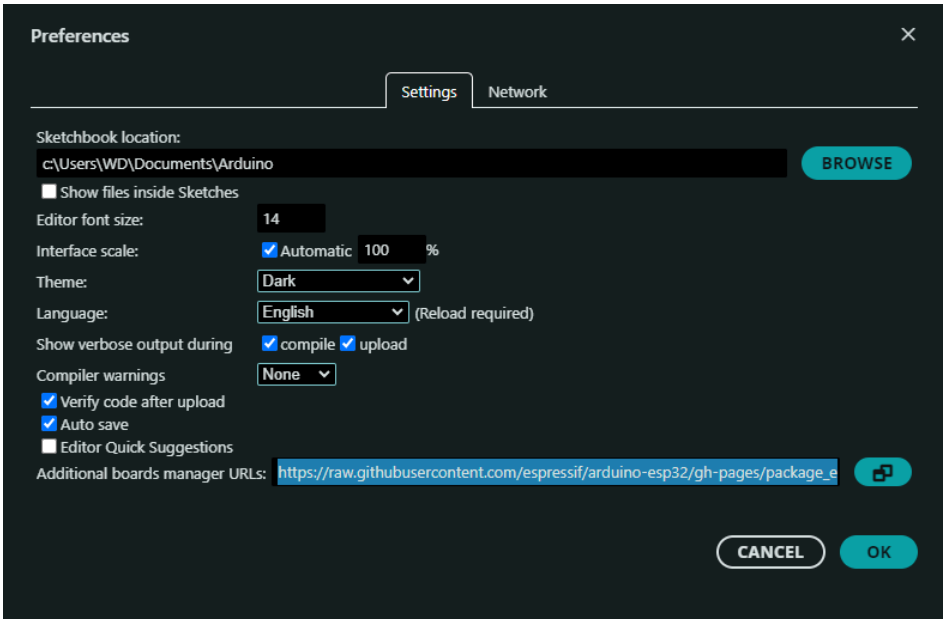
ข้อควรระวัง สาย USB ต้องเป็นชนิดที่รับส่งข้อมูลได้ สายสำหรับชาร์จไฟเพียงอย่างเดียวจะทำให้คอมพิวเตอร์มองไม่เห็นบอร์ด แม้ไฟเลี้ยงบนบอร์ดจะติดตามปกติ

ขั้นตอนที่ 3 เพิ่ม Board Manager URL ของ ESP32

1. เปิดโปรแกรม Arduino IDE
2. Windows / Linux: เลือกเมนู **File** → **Preferences** · macOS: เลือกเมนู **Arduino IDE** → **Settings**
3. ที่ช่อง **Additional boards manager URLs** ให้วาง URL ต่อไปนี้

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

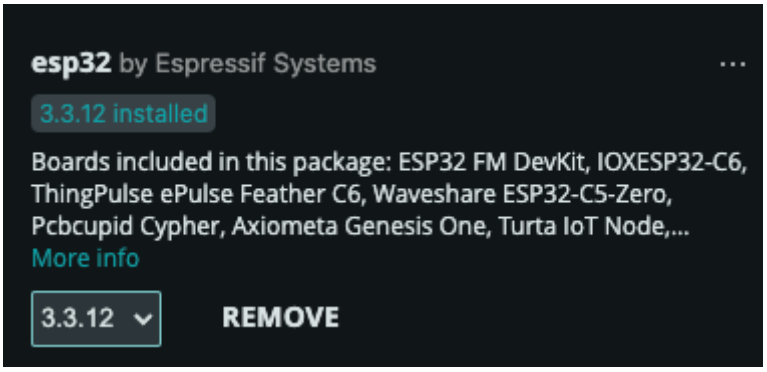
4. กด **OK**



ภาพที่ 7 หน้าต่าง Preferences ของ Arduino IDE ที่กรอก Board Manager URL แล้ว

ขั้นตอนที่ 4 ติดตั้งแพ็กเกจบอร์ด ESP32 เวอร์ชัน 3.x

1. เลือกเมนู **Tools** → **Board** → **Boards Manager** หรือคลิกไอคอนบอร์ดที่แถบด้านซ้าย
2. พิมพ์คำค้นหา esp32
3. เลือกรายการ **esp32 by Espressif Systems**
4. ในช่องเลือกเวอร์ชัน ให้เลือกเวอร์ชัน **3.x.x** (เวอร์ชันล่าสุดของชุด 3)
5. กด **INSTALL** และรอจนการติดตั้งเสร็จสมบูรณ์



ภาพที่ 8 หน้าต่าง Boards Manager แสดงการเลือก esp32 by Espressif Systems เวอร์ชัน 3.x.x

> **ข้อควรระวัง** หากติดตั้งเวอร์ชัน 2.x การคอมไพล์จะแจ้งข้อผิดพลาด 'ledcAttach' was not declared in this scope ให้กลับมาที่ Boards Manager แล้วเปลี่ยนเป็นเวอร์ชัน 3.x.x

ขั้นตอนที่ 5 ติดตั้งไลบรารี NimBLE-Arduino และ Massmore_MomoJoy

5.1 ดาวน์โหลดไฟล์ไลบรารี

1. เปิดเว็บไซต์ https://github.com/Massmore/Massmore_MOMOJoystick_SKU-1001
2. กดปุ่ม **Code** → **Download ZIP** แล้วแตกไฟล์
3. เปิดโฟลเดอร์ **ArduinoIDE** จะพบไฟล์สำหรับติดตั้ง 2 ไฟล์

ไฟล์	รายละเอียด	ลำดับการติดตั้ง
NimBLE-Arduino.zip	ไลบรารีสื่อสาร BLE ที่ไลบรารีหลักเรียกใช้	ลำดับที่ 1
Massmore_MomoJoy.zip	ไลบรารีหลัก พร้อมโปรแกรมตัวอย่าง 6 ชุด	ลำดับที่ 2

5.2 นำเข้าไลบรารีสู่ Arduino IDE

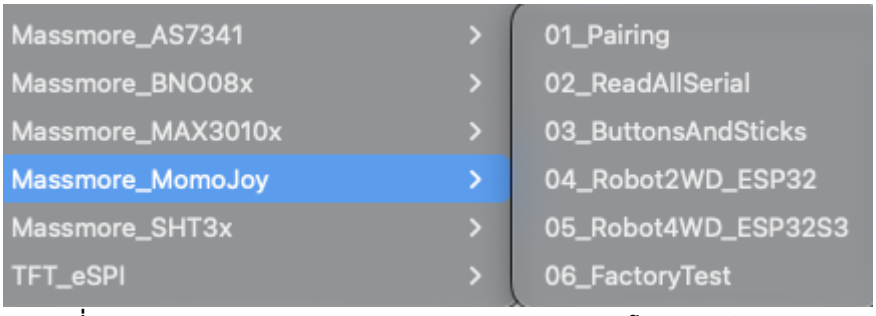
1. เลือกเมนู **Sketch** → **Include Library** → **Add .ZIP Library**
2. เลือกไฟล์ **NimBLE-Arduino.zip** แล้วกด **Open**
3. ทำซ้ำข้อ 1–2 โดยเลือกไฟล์ **Massmore_MomoJoy.zip**
4. ปิดโปรแกรม Arduino IDE แล้วเปิดใหม่หนึ่งครั้ง

ข้อควรระวัง หากเคยติดตั้งไลบรารีเวอร์ชันก่อนหน้า ให้ลบโฟลเดอร์ MomoJoy และ Massmore_MomoJoy ออกจากโฟลเดอร์ไลบรารีก่อนนำเข้าใหม่ มิฉะนั้นโปรแกรมจะยังแสดงตัวอย่างชุดเดิม ตำแหน่งโฟลเดอร์ไลบรารีคือ Windows: Documents\Arduino\libraries\ · macOS: ~/Documents/Arduino/libraries/ · Linux: ~/Arduino/libraries/

5.3 ตรวจสอบผลการติดตั้ง

เลือกเมนู **File → Examples** แล้วเลื่อนหาหัวข้อ **Massmore_MomoJoy** หากพบโปรแกรมตัวอย่างครบ 6 ชุดตามตาราง แสดงว่าการติดตั้งสำเร็จ

ลำดับ	ชื่อโปรแกรมตัวอย่าง	วัตถุประสงค์	บอร์ดที่ใช้	หัวข้อในคู่มือ
01	01_Pairing	จับคู่จอยครั้งแรกและแสดงสถานะการเชื่อมต่อ	ทั้งสองรุ่น	4.1
02	02_ReadAllSerial	แสดงค่าทุกปุ่มและทุกแกน และใช้สำหรับปรับเทียบ	ทั้งสองรุ่น	4.2
03	03_PairByAddress	ระบุจอยด้วย MAC Address เมื่อมีจอยหลายตัว	ทั้งสองรุ่น	4.3
04	04_Robot2WD_ESP32	หุ่นยนต์ 2 ล้อ	ESP32 Breakout (SKU-1015)	บทที่ 5
05	05_Robot4WD_ESP32S3	หุ่นยนต์ 4 ล้อ เคลื่อนที่รอบทิศทาง	MOMO ESP32-S3 (SKU-1020)	บทที่ 6
06	06_FactoryTest	ทดสอบจอยทีละปุ่มแบบมีคำแนะนำ	ทั้งสองรุ่น	บทที่ 7



ภาพที่ 9 เมนู File → Examples → Massmore_MomoJoy แสดงโปรแกรมตัวอย่าง 6 ชุด

ขั้นตอนที่ 6 ตั้งค่าบอร์ดในเมนู Tools

เลือกเมนู **Tools** แล้วตั้งค่าให้ตรงกับบอร์ดที่ใช้ ตามตารางต่อไปนี้ ค่าที่เป็นตัวหนาคือค่าที่ต้องตรวจสอบเป็นพิเศษ

6.1 บอร์ด MOMO ESP32-S3 (SKU-1020)

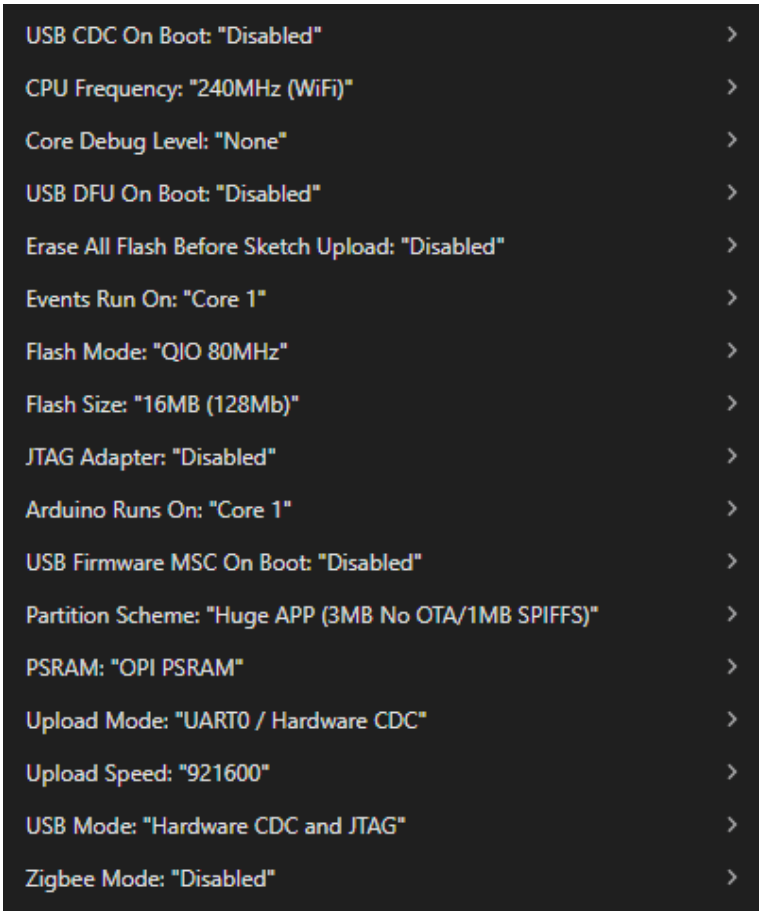
หัวข้อในเมนู Tools	ค่าที่ต้องเลือก	เหตุผล
Board	**ESP32S3 Dev Module**	ตรงกับชิป ESP32-S3 บนบอร์ด
Port	พอร์ตของชิป CH343P	พอร์ตที่ปรากฏหลังเสียบสาย USB
USB CDC On Boot	**Disabled**	ข้อความ Serial ส่งผ่านชิป CH343P ไม่ใช่พอร์ต USB ของ ESP32-S3
CPU Frequency	240MHz (WiFi)	ความเร็วสูงสุดของชิป
Flash Mode	QIO 80MHz	ค่ามาตรฐานของโมดูล
Flash Size	**16MB (128Mb)**	ตรงกับโมดูล N16R8
Partition Scheme	**16M Flash (3MB APP/9.9MB FATFS)**	มีพื้นที่โปรแกรมเพียงพอสำหรับ BLE
PSRAM	**OPI PSRAM**	ตรงกับ PSRAM 8 MB ชนิด Octal ของโมดูล
Upload Mode	UART0 / Hardware CDC	อัปโหลดผ่านชิป CH343P
Upload Speed	921600	ลดเป็น 460800 หากอัปโหลดไม่สำเร็จ
Core Debug Level	None	ลดข้อความที่ไม่จำเป็นใน Serial Monitor

ข้อควรระวัง หากตั้งค่า **USB CDC On Boot** เป็น Enabled หน้าต่าง Serial Monitor จะไม่แสดงข้อความใด ๆ แม้โปรแกรมจะทำงานตามปกติ

6.2 บอร์ด ESP32 Breakout (SKU-1015)

หัวข้อในเมนู Tools	ค่าที่ต้องเลือก	เหตุผล
Board	**ESP32 Dev Module**	ตรงกับโมดูล ESP32-WROOM-32
Port	พอร์ตของชิป USB-to-Serial บนโมดูล	พอร์ตที่ปรากฏหลังเสียบสาย USB
CPU Frequency	240MHz (WiFi/BT)	ความเร็วสูงสุดของชิป
Flash Frequency	80MHz	ค่ามาตรฐานของโมดูล
Flash Mode	QIO	ค่ามาตรฐานของโมดูล
Flash Size	**4MB (32Mb)**	ตรงกับโมดูล 38PIN ทั่วไป
Partition Scheme	**Huge APP (3MB No OTA/1MB SPIFFS)**	โปรแกรมที่ใช้ BLE มีขนาดเกินพื้นที่ 1.2 MB ของค่ามาตรฐาน
PSRAM	Disabled	โมดูลไม่มี PSRAM
Upload Speed	921600	ลดเป็น 460800 หากอัปโหลดไม่สำเร็จ
Core Debug Level	None	ลดข้อความที่ไม่จำเป็นใน Serial Monitor

ข้อควรระวัง หากไม่เลือก **Huge APP** การคอมไพล์จะแจ้งข้อผิดพลาด Sketch too big เนื่องจากพื้นที่โปรแกรมไม่เพียงพอ



ภาพที่ 10 เมนู Tools ที่ตั้งค่าสำหรับบอร์ด MOMO ESP32-S3 เรียบร้อยแล้ว

ขั้นตอนที่ 7 ทดสอบการอัปโหลดโปรแกรม

ขั้นตอนนี้ใช้ตรวจสอบว่าคอมพิวเตอร์ บอร์ด และไลบรารีพร้อมทำงานร่วมกัน ก่อนเริ่มใช้งานจอยจริง

1. เลือกเมนู **File → New Sketch**
2. ลบโค้ดเดิมทั้งหมด แล้วพิมพ์โค้ดต่อไปนี้

```
#include <Massmore_MomoJoy.h>
using namespace massmore;

void setup() {
 // กำหนดขาควบคุมมอเตอร์เป็น LOW ก่อน เพื่อป้องกันมอเตอร์หมุนเอง
#if CONFIG_IDF_TARGET_ESP32S3
 const uint8_t pins[] = {21, 38, 39, 40, 41, 42, 47, 48, 45};  // SKU-1020
#else
 const uint8_t pins[] = {27, 26, 16, 17, 19, 32, 33};         // SKU-1015
#endif
 for (uint8_t p : pins) { pinMode(p, OUTPUT); digitalWrite(p, LOW); }

 Serial.begin(115200);
 delay(500);
 Serial.println("Hello MOMO");
 Serial.printf("Chip    : %s\n", ESP.getChipModel());
 Serial.printf("Library : Massmore_MomoJoy v%s\n", MomoJoy.version());
}

void loop() {
}
```

3. กดปุ่ม **Verify** (เครื่องหมายถูก) เพื่อคอมไพล์ หากไม่มีข้อผิดพลาด ให้กดปุ่ม **Upload** (ลูกศรขวา)
4. เมื่ออัปโหลดเสร็จ เลือกเมนู **Tools → Serial Monitor** ตั้งค่าความเร็วที่มุมขวาเป็น **115200 baud** แล้วกดปุ่ม **RST** บนบอร์ดหนึ่งครั้ง

ผลลัพธ์ที่ถูกต้อง

```
Hello MOMO
Chip    : ESP32-S3
Library : Massmore_MomoJoy v2.1.0
```

สำหรับบอร์ด ESP32 Breakout บรรทัด Chip จะแสดงชื่อรุ่นของชิป ESP32 เช่น ESP32-D0WD-V3

> **หมายเหตุ** หากอัปโหลดไม่สำเร็จและแจ้งข้อความ Failed to connect ให้กดปุ่ม **BOOT** ค้างไว้ กดปุ่ม **RST** หนึ่งครั้ง ปล่อยปุ่ม **RST** แล้วจึงปล่อยปุ่ม **BOOT** จากนั้นกด **Upload** อีกครั้ง

ขั้นตอนที่ 8 ตรวจสอบความพร้อมตามรายการตรวจสอบ

ลำดับ	รายการตรวจสอบ	ผล
1	ติดตั้ง Arduino IDE 2.x เรียบร้อย	☐
2	คอมพิวเตอร์มองเห็นพอร์ตของบอร์ด	☐
3	ติดตั้งแพ็กเกจ esp32 by Espressif Systems เวอร์ชัน 3.x.x	☐
4	ติดตั้งไลบรารี NimBLE-Arduino และ Massmore_MomoJoy ครบทั้ง 2 ไฟล์	☐
5	พบโปรแกรมตัวอย่าง Massmore_MomoJoy ครบ 6 ชุด	☐
6	ตั้งค่าเมนู Tools ตรงกับบอร์ดที่ใช้	☐
7	อัปโหลดโปรแกรมทดสอบสำเร็จ และ Serial Monitor แสดงผลถูกต้อง	☐

เมื่อผ่านครบทุกรายการ ผู้ใช้งานพร้อมดำเนินการในบทที่ 4

4. การเชื่อมต่อและอ่านค่าจอยบังคับ (User Guide)

บทนี้ใช้ได้กับบอร์ดทั้งสองรุ่น แนะนำให้ดำเนินการตามลำดับหัวข้อ เพื่อให้แยกได้ชัดเจนว่าปัญหาที่อาจเกิดขึ้นภายหลังมาจากจอย การเชื่อมต่อ หรือโปรแกรม

4.1 การจับคู่จอยครั้งแรก (ตัวอย่าง 01_Pairing)

ขั้นตอนที่ 1 เปิดโปรแกรมตัวอย่าง

เลือกเมนู **File** → **Examples** → **Massmore_MomoJoy** → **01_Pairing**

ขั้นตอนที่ 2 อัปโหลดโปรแกรม

1. เสียบสาย USB ระหว่างคอมพิวเตอร์กับบอร์ด
2. ตรวจสอบการตั้งค่าในเมนู **Tools** ตามขั้นตอนที่ 6 ของบทที่ 3
3. กดปุ่ม **Upload**

ขั้นตอนที่ 3 เปิด Serial Monitor

เลือกเมนู **Tools** → **Serial Monitor** และตั้งค่าความเร็วเป็น **115200 baud**

ขั้นตอนที่ 4 เข้าสู่โหมดจับคู่ที่ตัวจอย

1. เลื่อน **สวิตช์โหมดไปที่ตำแหน่ง D**
2. กดปุ่ม **HOME ค้างไว้ประมาณ 3 วินาที** จนไฟแสดงสถานะ **กะพริบถี่** (หากไฟติดค้าง แสดงว่ายังไม่เข้าสู่โหมดจับคู่)
3. กดปุ่ม **RST** บนบอร์ดหนึ่งครั้ง บอร์ดจะค้นหาและเชื่อมต่อกับจอยเอง



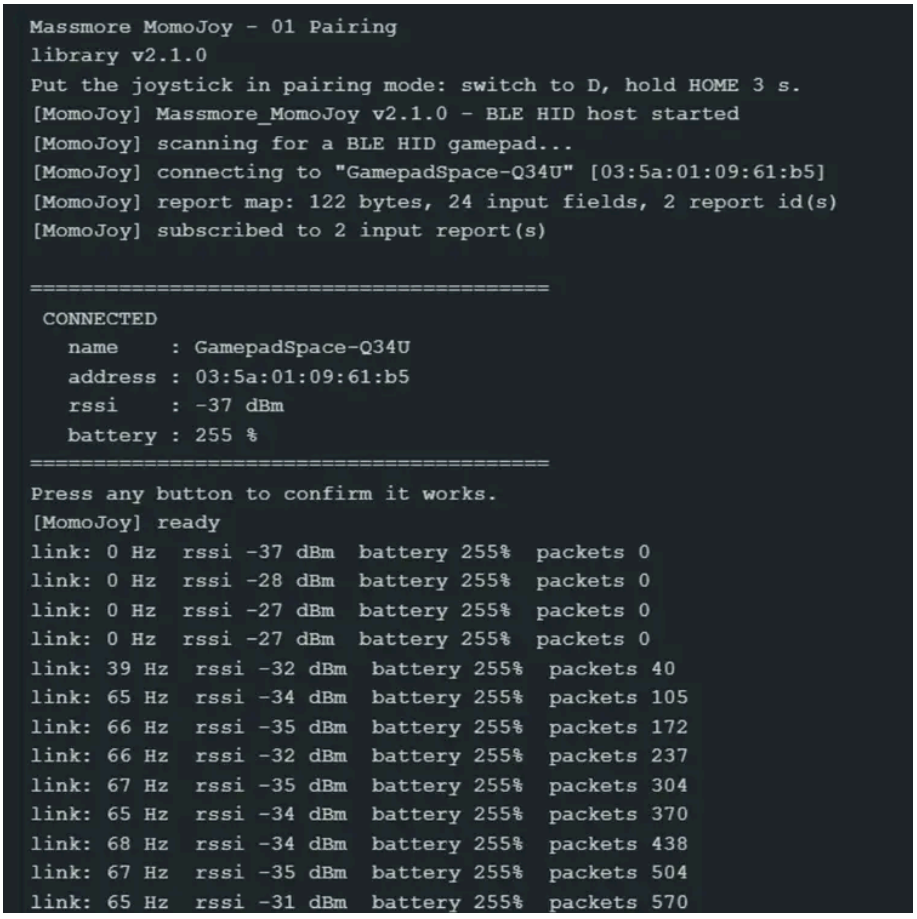
ภาพที่ 11 ขั้นตอนเข้าสู่โหมดจับคู่: (ก) ตำแหน่งสวิตช์โหมด D (ข) ปุ่ม HOME และไฟแสดงสถานะที่กะพริบถี่

ขั้นตอนที่ 5 ตรวจสอบผลใน Serial Monitor

เมื่อเชื่อมต่อสำเร็จ หน้าต่าง Serial Monitor จะแสดงข้อความในลักษณะต่อไปนี้

```
Massmore MomoJoy - 01 Pairing
library v2.1.0
Put the joystick in pairing mode: switch to D, hold HOME 3 s.
========================================
 CONNECTED
  name   : <ชื่อที่จอยประกาศ>
  address : xx:xx:xx:xx:xx:xx
  rssi   : -41 dBm
  battery : 87 %
========================================
Press any button to confirm it works.
button: A
link: 66 Hz  rssi -42 dBm  battery 87%  packets 1520
```

เมื่อกดปุ่มบนจอย จะปรากฏบรรทัด button: ตามด้วยชื่อปุ่ม และทุก 1 วินาทีจะปรากฏบรรทัด link: แสดงอัตราการส่งข้อมูล ความแรงสัญญาณ ระดับแบตเตอรี่ และจำนวนแพ็กเก็ตที่ได้รับ



ภาพที่ 12 หน้าต่าง Serial Monitor แสดงผลการเชื่อมต่อสำเร็จจากตัวอย่าง 01_Pairing

ข้อแนะนำ ให้บันทึก MAC Address ของจอย (ค่าในบรรทัด address) ไว้ เพื่อใช้ในหัวข้อ 4.3 กรณีใช้งานจอยหลายตัว

ขั้นตอนที่ 6 การเชื่อมต่อในครั้งถัดไป

ข้อมูลการจับคู่ถูกบันทึกในหน่วยความจำ NVS ของบอร์ด เมื่อเปิดบอร์ดและจอยในครั้งถัดไป ระบบจะเชื่อมต่อให้อัตโนมัติโดยไม่ต้องจับคู่ใหม่

หากต้องการล้างข้อมูลการจับคู่ เช่น เพื่อเปลี่ยนไปใช้จอยตัวอื่น ให้ **กดปุ่ม SELECT และ START ค้างพร้อมกันประมาณ 1.5 วินาที** ขณะที่จอยยังเชื่อมต่ออยู่ แล้วจับคู่ใหม่ตามขั้นตอนที่ 4

4.2 การอ่านค่าทั้งหมดจากจอย (ตัวอย่าง 02_ReadAllSerial)

โปรแกรมตัวอย่างนี้แสดงค่าของก้าน ไก D-Pad ปุ่ม และระดับแบตเตอรี่ทางหน้าต่าง Serial Monitor โดยแสดงเฉพาะเมื่อค่ามีการเปลี่ยนแปลง จึงอ่านผลได้ง่าย เหมาะสำหรับการเรียนรู้ความหมายของค่าแต่ละแกน และใช้เป็นเครื่องมือปรับเทียบปุ่ม

ขั้นตอนการใช้งาน

1. เลือกเมนู **File → Examples → Massmore_MomoJoy → 02_ReadAllSerial**
2. เลือกรูปแบบการแสดงผลที่บรรทัดด้านบนของโปรแกรม แล้วอัปโหลด
3. เปิด Serial Monitor ที่ความเร็ว **115200 baud** แล้วทดลองกดปุ่มและโยกก้านทีละรายการ

```
#define SHOW_RAW_REPORTS 0    // 1 = แสดงข้อมูลดิบของ HID สำหรับการปรับเทียบ
#define OUTPUT_FORMAT    0    // 0 = ข้อความ, 1 = CSV, 2 = JSON
```

ค่า OUTPUT_FORMAT	รูปแบบ	การนำไปใช้
0	ข้อความสำหรับอ่าน	การเรียนรู้และตรวจสอบค่าด้วยสายตา
1	CSV ลำดับคอลัมน์ lx,ly,rx,ry,l2,r2,dpad,buttons,battery	บันทึกข้อมูลเพื่อนำไปวิเคราะห์หรือเขียนกราฟ
2	JSON หนึ่งบรรทัดต่อหนึ่งชุดข้อมูล	ส่งต่อให้โปรแกรมบนคอมพิวเตอร์หรือเว็บไซต์

ข้อแนะนำ ในการเรียนการสอน สามารถใช้รูปแบบ CSV ร่วมกับเครื่องมือ **Tools → Serial Plotter** ของ Arduino IDE เพื่อแสดงกราฟค่าก้านแบบเวลาจริงได้

4.3 การใช้งานจอยหลายตัวในพื้นที่เดียวกัน (ตัวอย่าง 03_PairByAddress)

โดยค่าเริ่มต้น ไลบรารีจะเชื่อมต่อกับ **จอยตัวแรกที่ตรวจพบ** หากเปิดจอยหลายตัวพร้อมกัน เช่น ในห้องเรียนหรือสายการผลิต บอร์ดอาจเชื่อมต่อกับจอยผิดตัว วิธีแก้ไขคือการระบุจอยด้วย **MAC Address** โดยดำเนินการ 2 รอบ

ขั้นตอนที่ 1 ค้นหา MAC Address ของจอย

1. เลือกเมนู **File → Examples → Massmore_MomoJoy → 03_PairByAddress**
2. คงค่าบรรทัดต่อไปนี้ไว้เป็นค่าว่างตามเดิม แล้วอัปโหลด

```
#define TARGET_ADDRESS ""
```

3. เปิด Serial Monitor บอร์ดจะสแกนประมาณ 6 วินาที แล้วแสดงรายชื่อจอยทุกตัวในระยะ

```
========================================
2 gamepad(s) in range
 #  rssi   address          name
 ----------------------------------------
 1  -41   aa:bb:cc:dd:ee:01  <ชื่อจอยตัวที่ 1>
 2  -73   aa:bb:cc:dd:ee:02  <ชื่อจอยตัวที่ 2>
========================================
```

คอลัมน์ **rssi** แสดงความแรงสัญญาณ ค่าที่ใกล้ 0 หมายถึงจอยอยู่ใกล้บอร์ดมากกว่า ใช้ประกอบการระบุว่าจอยตัวใดเป็นของผู้ใช้งาน

ขั้นตอนที่ 2 กำหนด MAC Address แล้วอัปโหลดใหม่

```
#define TARGET_ADDRESS "aa:bb:cc:dd:ee:01"
```

หลังอัปโหลด บอร์ดจะเชื่อมต่อกับจอยตัวที่กำหนดเพียงตัวเดียว และเชื่อมต่อให้เองทุกครั้งที่เปิดเครื่องโดยไม่ต้องกดปุ่มใด

ทางเลือกสำหรับโต๊ะปฏิบัติการที่มีจอยเพียงตัวเดียว

กำหนดค่าต่อไปนี้เป็น 1 บอร์ดจะเลือกจอยที่มีสัญญาณแรงที่สุดให้อัตโนมัติ

```
#define AUTO_PICK_STRONGEST 1
```

การนำไปใช้ในโปรแกรมของผู้ใช้งาน

```
MomoJoyOptions opt;
opt.addressFilter = "aa:bb:cc:dd:ee:01";   // เชื่อมต่อกับจอยตัวนี้เท่านั้น
// opt.nameFilter = "MOMO";              // หรือกรองด้วยชื่อที่จอยประกาศ
MomoJoy.begin(opt);
```

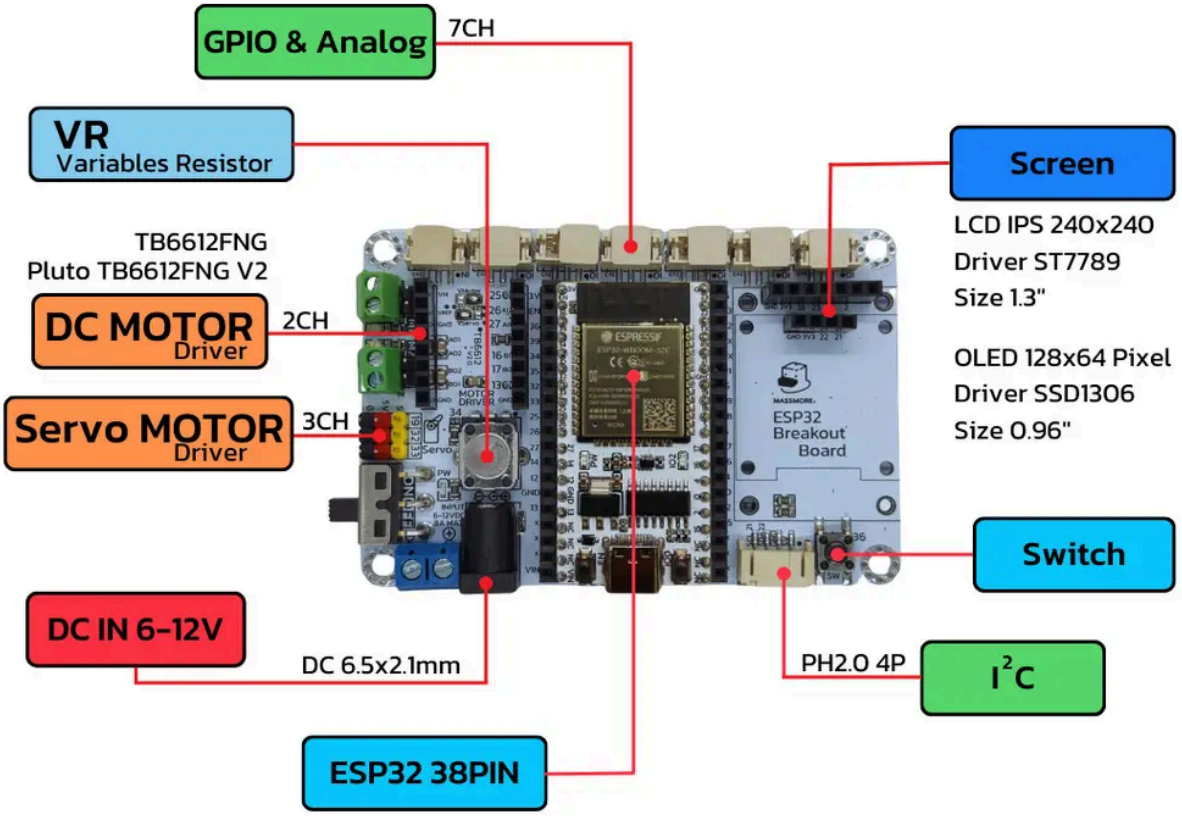
คำสั่ง	หน้าที่
opt.autoStart = false	ให้ begin() เตรียมระบบ BLE โดยยังไม่เชื่อมต่อกับจอยตัวแรกที่พบ
scanDevices(ms)	สแกนโดยไม่เชื่อมต่อ คืนค่าจำนวนจอยที่พบ (สูงสุด 8 ตัว)
foundDevice(i)	อ่านข้อมูล .name .address .rssi ของจอยลำดับที่ i
setAddressFilter(mac)	เปลี่ยน MAC Address ที่ต้องการขณะโปรแกรมทำงาน
connectToAddress(mac)	กำหนด MAC Address และเริ่มค้นหาในคำสั่งเดียว

ข้อแนะนำ สำหรับห้องปฏิบัติการ แนะนำให้ติดป้ายหมายเลขที่จอยและบอร์ดให้ตรงกัน พร้อมบันทึก MAC Address ของแต่ละคู่ไว้ในใบบันทึกประจำโต๊ะ เพื่อความสะดวกในการจัดการเมื่อใช้งานจำนวนมาก

5. การใช้งานกับบอร์ด ESP32 Breakout (SKU-1015)

5.1 ข้อมูลบอร์ด

บอร์ด ESP32 Breakout เป็นบอร์ดขยายสำหรับโมดูล **ESP32 38PIN** มีวงจรแปลงแรงดันไฟ จุดต่อสัญญาณควบคุมมอเตอร์ 2 ช่อง ช่องเซอร์โว 3 ช่อง ตัวต้านทานปรับค่า (VR) และสวิตช์บนบอร์ด เหมาะสำหรับหุ่นยนต์ขับเคลื่อน 2 ล้อ



ภาพที่ 13 บอร์ด ESP32 Breakout (SKU-1015) ระบุตำแหน่งขั้วไฟเลี้ยง ช่องโมดูลขับมอเตอร์ ช่องมอเตอร์ M1/M2 ช่องเซอร์โว VR และ SW

5.1.1 ผังขาที่โปรแกรมตัวอย่างใช้งาน

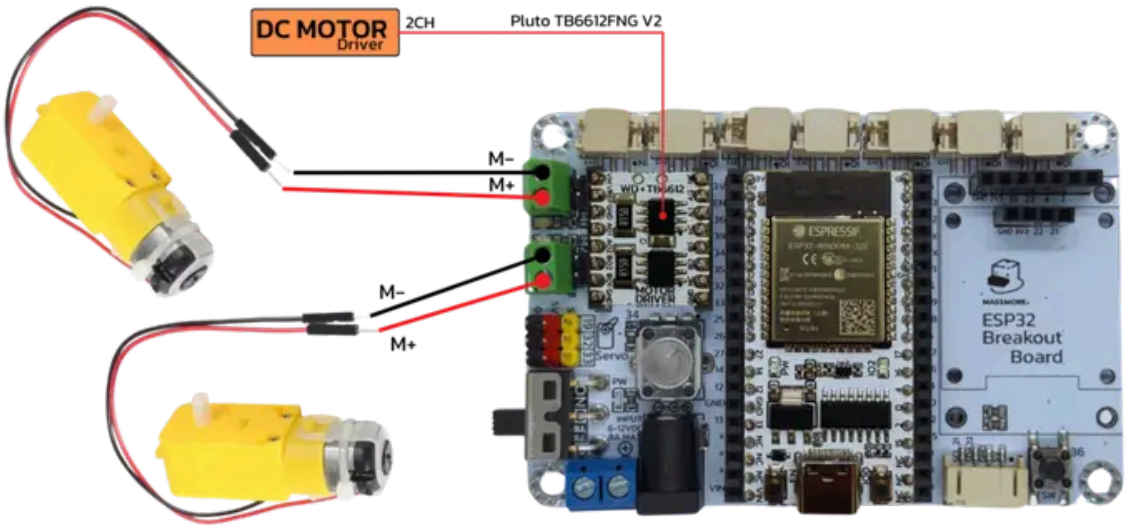
หน้าที่	สัญญาณ	GPIO	หมายเหตุ
มอเตอร์ซ้าย M1	IN1 / IN2	**27 / 26**	ส่งสัญญาณไปยังโมดูลขับมอเตอร์ภายนอก
มอเตอร์ขวา M2	IN1 / IN2	**16 / 17**	ส่งสัญญาณไปยังโมดูลขับมอเตอร์ภายนอก
เซอร์โว 1 / 2 / 3	Servo	**19 / 32 / 33**	สัญญาณ PWM 50 Hz
ตัวต้านทานปรับค่า	VR	**34**	อินพุตอนาล็อก ใช้ตั้งความเร็วสูงสุด
สวิตช์บนบอร์ด	SW	**36**	ขาอินพุตเท่านั้น
บัส I2C	SDA / SCL	21 / 22	สำหรับจอ OLED และเซนเซอร์
จอแสดงผล SPI	CLK / MOSI / RES / DC	18 / 23 / 4 / 2	ไม่ได้ใช้ในตัวอย่าง
ขาอเนกประสงค์	IO1–IO7	25 / 13 / 12 / 14 / 15 / 5 / 35	สำรองสำหรับต่อขยาย

ข้อควรระวัง บอร์ด SKU-1015 **ไม่มีวงจรขับมอเตอร์บนบอร์ด** ขา IN1 / IN2 เป็นสัญญาณควบคุมที่ส่งไปยังโมดูลขับมอเตอร์ภายนอก (TB6612FNG หรือเทียบเท่า) ต้องติดตั้งโมดูลขับมอเตอร์ก่อน จึงจะขับมอเตอร์ได้ ห้ามต่อมอเตอร์เข้ากับขา GPIO โดยตรง

5.2 อุปกรณ์ที่ต้องใช้

ลำดับ	อุปกรณ์	จำนวน
1	MOMO Joystick Controller (SKU-1001)	1 ตัว
2	บอร์ด ESP32 Breakout (SKU-1015) พร้อมโมดูล ESP32 38PIN	1 ชุด
3	โมดูลขับมอเตอร์ TB6612FNG หรือเทียบเท่า	1 โมดูล
4	มอเตอร์เกียร์ DC พร้อมล้อ	2 ชุด
5	แบตเตอรี่แรงดัน 6–12 V	1 ก้อน
6	เซอร์โวมอเตอร์ (ถ้าต้องการทดลองปุ่ม A)	1 ตัว
7	สาย USB ชนิดรับส่งข้อมูลได้	1 เส้น

5.3 การต่อวงจร



ภาพที่ 14 แผนผังการต่อวงจรหุ่นยนต์ 2 ล้อ: แบตเตอรี่ → บอร์ด SKU-1015 → โมดูลขับมอเตอร์ → มอเตอร์ซ้าย (M1) และมอเตอร์ขวา (M2)

1. ปิดสวิตช์ไฟเลี้ยงทั้งหมดก่อนเริ่มต่อวงจร
2. ติดตั้งโมดูล ESP32 38PIN และโมดูลขับมอเตอร์ลงบนบอร์ดให้ถูกทิศทาง
3. ต่อมอเตอร์ล้อซ้ายเข้าช่อง **M1** และมอเตอร์ล้อขวาเข้าช่อง **M2**

4. ต่อเซอร์โวเข้าช่อง **Servo1** (ถ้ามี) โดยตรวจสอบตำแหน่งสายสัญญาณ ไฟเลี้ยง และกราวด์
5. ต่อแบตเตอรี่แรงดัน **6–12 V** เข้าขั้ว VIN ของบอร์ด โดยตรวจสอบขั้วบวกและลบให้ถูกต้อง
6. ยกล้อหุ่นยนต์ให้พ้นพื้นก่อนทดสอบครั้งแรก

5.4 การอัปโหลดโปรแกรมตัวอย่าง 04_Robot2WD_ESP32

1. เลือกเมนู **File → Examples → Massmore_MomoJoy → 04_Robot2WD_ESP32**
2. ตั้งค่าเมนู **Tools** ตามตารางในบทที่ 3 ขั้นตอนที่ 6.2 โดยเฉพาะ **Partition Scheme = Huge APP**
3. กดปุ่ม **Upload** แล้วเปิด Serial Monitor ที่ความเร็ว **115200 baud**
4. หากยังไม่เคยจับคู่จอยกับบอร์ดนี้ ให้ทำตามหัวข้อ 4.1 ขั้นตอนที่ 4

5.5 วิธีการบังคับ

ปุ่ม / แกน	การทำงาน
ก้านซ้าย	ควบคุมการเคลื่อนที่ทั้งหมดด้วยก้านเดียว ดันขึ้น/ลง = เดินหน้า/ถอยหลัง ดันซ้าย/ขวา = เลี้ยว ความเร็วแปรผันตามระยะที่ดัน
D-Pad	เคลื่อนที่ตามทิศของปุ่มด้วยความเร็วคงที่
R2	เพิ่มความเร็วจากค่าที่ก้านกำหนด ตามระยะที่กดไก
Y (กดค้าง)	โหมดความเร็วสูง (Turbo)
A	ปรับมุมเซอร์โว 1 ตามลำดับ 0° → 90° → 180°
L1 / R1 (กดค้าง)	ทดสอบหมุนล้อซ้าย / ล้อขวาทีละข้าง
B	หยุดฉุกเฉิน
HOME (กดค้าง 2 วินาที)	ล้างข้อมูลการจับคู่ BLE
VR บนบอร์ด	กำหนดความเร็วสูงสุดในช่วง 70–200 (จาก 255) ปรับได้ทันทีโดยไม่ต้องแก้ไขโปรแกรม

5.6 การปรับค่าการตอบสนอง

ค่าต่อไปนี้อยู่ในส่วน #define ด้านบนของโปรแกรมตัวอย่าง

ชื่อค่า	ค่าเริ่มต้น	ความหมายและแนวทางการปรับ
SPEED_MIN / SPEED_MAX	70 / 200	ช่วงความเร็วสูงสุดที่ปรับด้วย VR
SPEED_TURBO	255	ความเร็วขณะกดปุ่ม Y ค้าง
SPEED_DPAD	110	ความเร็วคงที่เมื่อใช้ D-Pad
SPEED_TEST	150	ความเร็วขณะทดสอบล้อด้วย L1 / R1
MOTOR_MIN_DUTY	40	ค่าต่ำสุดที่มอเตอร์เริ่มหมุน หากหุ่นยนต์สั่นแต่ไม่เคลื่อนที่ ให้เพิ่มเป็น 50–60
EXPO_AMOUNT	0.60	ความนุ่มนวลช่วงกลางก้าน ค่ามากขึ้นจะควบคุมความเร็วต่ำได้ละเอียดขึ้น
TURN_GAIN	0.65	ความไวในการเลี้ยว

ชื่อค่า	ค่าเริ่มต้น	ความหมายและแนวทางการปรับ
SLEW_UP / SLEW_DOWN	6 / 24	อัตราการเร่งและการลดความเร็วต่อรอบควบคุม (รอบละ 20 มิลลิวินาที)

5.7 โครงสร้างโปรแกรมส่วนสำคัญ

```
void setup() {
  parkMotorPins();        // 1) กำหนดขามอเตอร์เป็น LOW เป็นคำสั่งแรก
  Serial.begin(115200);
  motorsBegin();          //   ตั้งค่า PWM 20 kHz ให้ขามอเตอร์

  MomoJoyOptions opt;
  opt.deadzone = 40;      //   งานหุ่นยนต์ใช้ Dead Zone กว้างกว่างานเกม
  MomoJoy.begin(opt);
}

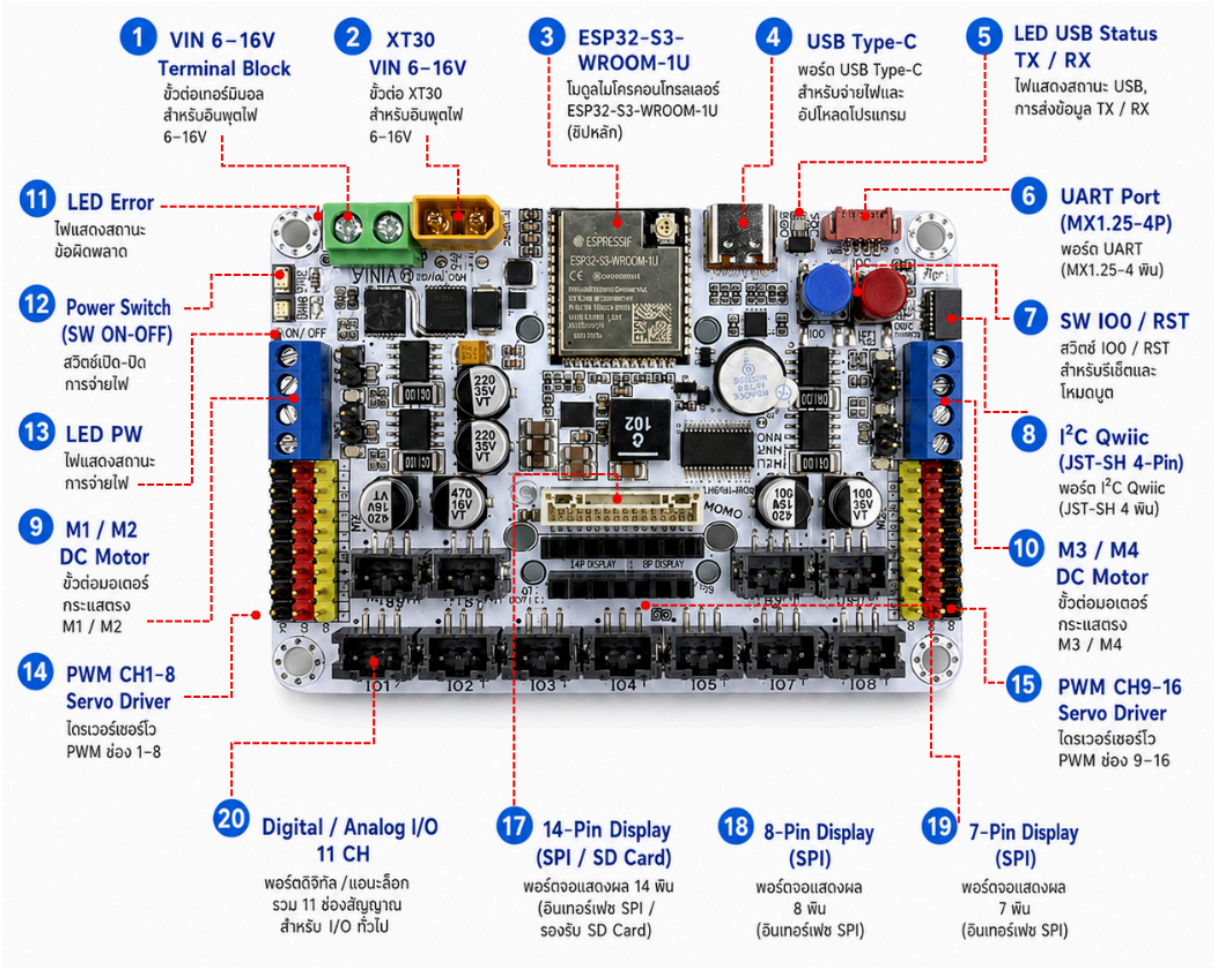
void loop() {
  MomoJoy.update();
  // 2) ระบบความปลอดภัย: ไม่มีสัญญาณ หรือไม่ได้รับข้อมูลเกิน 400 ms ให้หยุดทันที
  if (!MomoJoy.isConnected() || MomoJoy.msSinceReport() > 400) {
    motorsStop();
    return;
  }
  // 3) ก้านซ้าย: แกน Y = เดินหน้า/ถอยหลัง, แกน X = เลี้ยว
  int maxSpeed = MomoJoy.pressed(MOMO_BTN_Y) ? SPEED_TURBO : knobSpeed();
  // ... คำนวณความเร็วล้อซ้าย/ขวา แล้วสั่งงานมอเตอร์
}
```

โค้ดข้างต้นเป็นโครงสร้างโดยย่อเพื่อการอธิบาย รายละเอียดฉบับเต็มอยู่ในไฟล์ 04_Robot2WD_ESP32.ino

6. การใช้งานกับบอร์ด MOMO ESP32-S3 (SKU-1020)

6.1 ข้อมูลบอร์ด

บอร์ด MOMO ESP32-S3 เป็นบอร์ดพัฒนาหุ่นยนต์ที่มี **วงจรขับมอเตอร์ TB67H451AFNG จำนวน 4 ช่อง** บนบอร์ด รองรับ Encoder ของมอเตอร์ทั้ง 4 ตัว มี Buzzer และช่องต่อจอแสดงผล เหมาะสำหรับหุ่นยนต์ 4 ล้อ รวมถึงหุ่นยนต์ล้อ Mecanum ที่เคลื่อนที่ได้รอบทิศทาง



ภาพที่ 15 บอร์ด MOMO ESP32-S3 (SKU-1020) ระบุตำแหน่งขั้วไฟเลี้ยง (Terminal / XT30) สวิตช์ Power ช่องมอเตอร์ M1–M4 IO Header ปุ่ม BOOT/RST และพอร์ต USB

6.1.1 ผังขาที่โปรแกรมตัวอย่างใช้งาน

มอเตอร์	IN1	IN2	Encoder A / B	ตำแหน่งล้อในโปรแกรมตัวอย่าง
M1	**GPIO 21**	**GPIO 38**	GPIO 1 / 2	หลังขวา (Rear-Right)
M2	**GPIO 39**	**GPIO 40**	GPIO 16 / 17	หน้าขวา (Front-Right)
M3	**GPIO 41**	**GPIO 42**	GPIO 7 / 8	หน้าซ้าย (Front-Left)
M4	**GPIO 47**	**GPIO 48**	GPIO 9 / 10	หลังซ้าย (Rear-Left)
Buzzer	**GPIO 45**	—	—	Buzzer บนบอร์ด

6.1.2 การเทียบชื่อขาบน IO Header สำหรับสาย Encoder

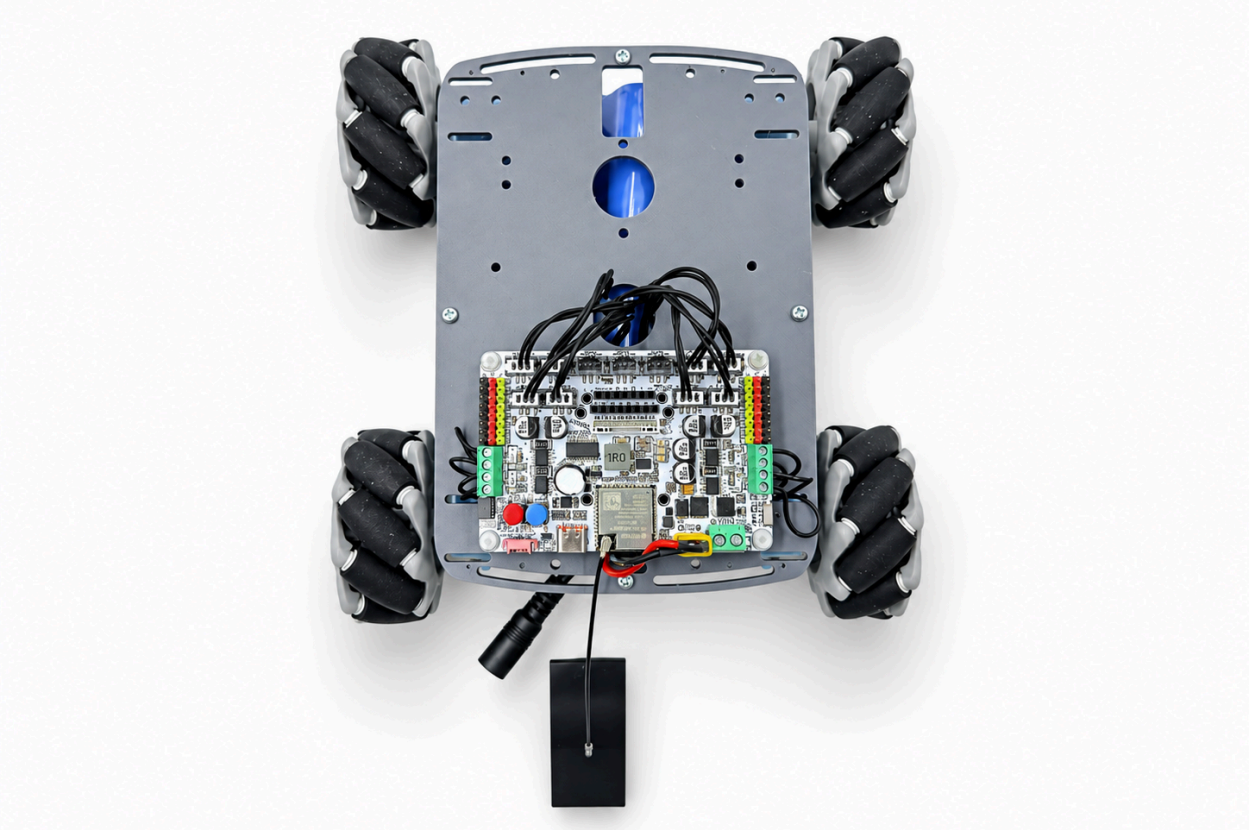
ชื่อบนบอร์ด	GPIO	ใช้งานเป็น
IO1 / IO2	1 / 2	Encoder M1 A / B
IO10 / IO11	16 / 17	Encoder M2 A / B
IO6 / IO7	7 / 8	Encoder M3 A / B
IO8 / IO9	9 / 10	Encoder M4 A / B

ข้อควรระวัง ขา **IO11 (GPIO 17)** ใช้ร่วมกับสัญญาณ TFT_CS ของจอแสดงผล หากโครงงานต้องใช้จอแสดงผลด้วย ห้ามใช้ IO11 เป็นสาย Encoder พร้อมกัน ให้ย้าย Encoder ของ M2 ไปใช้ขาอื่น แล้วแก้ไขหมายเลขขาในโปรแกรม

6.2 อุปกรณ์ที่ต้องใช้

ลำดับ	อุปกรณ์	จำนวน
1	MOMO Joystick Controller (SKU-1001)	1 ตัว
2	บอร์ด MOMO ESP32-S3 (SKU-1020)	1 บอร์ด
3	มอเตอร์เกียร์ DC พร้อม Encoder และล้อ (ล้อ Mecanum หรือล้อปกติ)	4 ชุด
4	แบตเตอรี่แรงดัน 6–16 V พร้อมขั้วต่อ XT30 หรือสายสำหรับ Terminal Block	1 ก้อน
5	สาย USB ชนิดรับส่งข้อมูลได้ (ตามชนิดพอร์ตของบอร์ด)	1 เส้น

6.3 การต่อวงจร



ภาพที่ 16 แผนผังการต่อวงจรหุ่นยนต์ 4 ล้อ: แบตเตอรี่ (XT30) → บอร์ด, มอเตอร์ M1–M4 ตามตำแหน่งล้อ, สาย Encoder → IO Header และทิศทางการวางล้อ Mecanum เมื่อมองจากด้านบน

1. ปิดสวิตช์ Power บนบอร์ดก่อนเริ่มต่อวงจร
2. ต่อมอเตอร์ 4 ตัวเข้าช่อง **M1–M4** ตามตำแหน่งล้อในตารางหัวข้อ 6.1.1
3. ต่อสาย Encoder เข้า **IO Header** ตามตารางหัวข้อ 6.1.2
4. ต่อแบตเตอรี่แรงดัน **6–16 V** เข้าขั้ว **Terminal Block** หรือ **XT30** โดยตรวจสอบขั้วให้ถูกต้อง
5. **เปิดสวิตช์ Power บนบอร์ด** และยกล้อหุ่นยนต์ให้พ้นพื้นก่อนทดสอบครั้งแรก

ข้อควรระวัง ต้องเปิดสวิตช์ Power บนบอร์ดทุกครั้งที่ต้องการขับมอเตอร์ หากปิดสวิตช์ไว้ บอร์ดจะได้รับไฟจากพอร์ต USB เพียงอย่างเดียว ซึ่งโปรแกรมทำงานได้แต่ไม่เพียงพอสำหรับขับมอเตอร์ อาการที่พบคือมอเตอร์ไม่หมุนแม้ต่อวงจรถูกต้องทั้งหมด

6.4 การอัปโหลดโปรแกรมตัวอย่าง 05_Robot4WD_ESP32S3

1. เลือกเมนู **File → Examples → Massmore_MomoJoy → 05_Robot4WD_ESP32S3**
2. ตั้งค่าเมนู **Tools** ตามตารางในบทที่ 3 ขั้นตอนที่ 6.1 โดยเฉพาะ **USB CDC On Boot = Disabled**
3. กดปุ่ม **Upload** แล้วเปิด Serial Monitor ที่ความเร็ว **115200 baud**
4. หากยังไม่เคยจับคู่จอยกับบอร์ดนี้ ให้ทำตามหัวข้อ 4.1 ขั้นตอนที่ 4

6.5 วิธีการบังคับ

ปุ่ม / แกน	การทำงาน
ก้านซ้าย	ดันไปทิศใด หุ่นยนต์เคลื่อนที่ไปทิศนั้น ทั้งเดินหน้า ถอยหลัง เคลื่อนที่ด้านข้าง และแนวทแยง ความเร็วแปรผันตามระยะที่ดัน
ก้านขวา แกน X	หมุนตัวอยู่กับที่ และใช้ร่วมกับการเคลื่อนที่ของก้านซ้ายได้
D-Pad	เคลื่อนที่ตามทิศของปุ่มด้วยความเร็วคงที่
Y (กดค้าง)	โหมดความเร็วสูง (Turbo)
A	เปิด/ปิดระบบรักษาความเร็ว (Cruise Control)
X	เคลื่อนที่ไปข้างหน้าอัตโนมัติประมาณ 1 เมตร โดยนับระยะจาก Encoder
B	หยุดฉุกเฉิน และยกเลิก Cruise Control และการเคลื่อนที่อัตโนมัติ
R3 (กดค้าง)	เปิดเสียง Buzzer
L1 / L2 / R1 / R2 (กดค้าง)	ทดสอบหมุนล้อหน้าซ้าย / หลังซ้าย / หน้าขวา / หลังขวา ทีละล้อ
SELECT / START	แสดงค่า Encoder / รีเซ็ตค่า Encoder เป็นศูนย์
HOME (กดค้าง 2 วินาที)	ล้างข้อมูลการจับคู่ BLE

6.6 การกำหนดชนิดล้อ

โปรแกรมตัวอย่างกำหนดค่าเริ่มต้นเป็น **ล้อ Mecanum** หากหุ่นยนต์ใช้ล้อปกติ ให้แก้ไขค่าต่อไปนี้เป็น 0 ก้านซ้ายจะควบคุมเฉพาะการเดินหน้าและถอยหลัง

```
#define MECANUM_WHEELS   0    // 1 = ล้อ Mecanum, 0 = ล้อปกติ
```

สมการผสมกำลังล้อ Mecanum ที่โปรแกรมตัวอย่างใช้ เป็นสมการมาตรฐานดังนี้

```
ล้อหน้าซ้าย  (FL) = fwd + strafe + rot
ล้อหน้าขวา  (FR) = fwd - strafe - rot
ล้อหลังซ้าย (RL) = fwd - strafe + rot
ล้อหลังขวา  (RR) = fwd + strafe - rot
```

โดย fwd คือค่าเดินหน้า strafe คือค่าเคลื่อนที่ด้านข้าง และ rot คือค่าหมุนตัว เมื่อคำนวณแล้วมีล้อใดเกินค่าสูงสุด โปรแกรมจะลดสเกลของทั้ง 4 ล้อในอัตราส่วนเดียวกัน เพื่อรักษาทิศทางการเคลื่อนที่ไม่ให้เบี่ยงเบน

> **ข้อแนะนำ** หากล้อใดหมุนกลับทิศ ให้สลับสายมอเตอร์ 2 เส้นของล้อนั้น หรือสลับหมายเลขขา IN1 / IN2 ของล้อนั้นในโปรแกรม โดยไม่ต้องแก้ไขสมการ

6.7 การปรับค่าการตอบสนอง

มอเตอร์เกียร์มีแรงบิดสูง โปรแกรมตัวอย่างจึงมีการหน่วงการเร่งและปรับเส้นโค้งการตอบสนองของก้าน ค่าต่อไปนี้อยู่ในส่วน #define ด้านบนของโปรแกรม

ชื่อค่า	ค่าเริ่มต้น	ความหมายและแนวทางการปรับ
SPEED_NORMAL	110 (จาก 255)	ความเร็วสูงสุดของก้านในโหมดปกติ
SPEED_TURBO	220	ความเร็วขณะกดปุ่ม Y ค้าง
SPEED_DPAD	110	ความเร็วคงที่เมื่อใช้ D-Pad
MOTOR_MIN_DUTY	40	ค่าต่ำสุดที่มอเตอร์เริ่มหมุน หากหุ่นยนต์สั่นแต่ไม่เคลื่อนที่ ให้เพิ่มเป็น 50–60
EXPO_AMOUNT	0.60	ความนุ่มนวลช่วงกลางก้าน เพิ่มเป็น 0.75 หากต้องการควบคุมความเร็วต่ำให้ละเอียดขึ้น
ROT_GAIN	0.60	ความไวในการหมุนตัว
SLEW_UP	6	อัตราการเร่ง ค่าน้อยลงจะออกตัวนุ่มนวลขึ้น
SLEW_DOWN	24	อัตราการลดความเร็ว
STICK_DEADZONE	40	ระยะที่ไม่ตอบสนองบริเวณกลางก้าน
PULSES_PER_METER	660	จำนวนพัลส์ Encoder ต่อระยะ 1 เมตร (ควรวัดจากหุ่นยนต์จริง)
AUTO_DRIVE_PULSES	615	ระยะเป้าหมายของปุ่ม X หน่วยพัลส์

ความเร็วของมอเตอร์ถูกแปลงจากระยะการดันก้านแบบต่อเนื่อง เริ่มจาก MOTOR_MIN_DUTY ไปจนถึงความเร็วสูงสุด จึงไม่มีช่วงที่มอเตอร์ไม่ตอบสนองหรือกระตุก ตัวอย่างเมื่อ SPEED_NORMAL = 110

ระยะการดันก้าน	5%	10%	25%	50%	75%	100%
ค่า PWM ที่ได้ (จาก 255)	41	43	48	59	79	110

7. การทดสอบจอยบังคับ (Factory Test)

7.1 วัตถุประสงค์

โปรแกรมตัวอย่าง 06_FactoryTest ใช้ตรวจสอบสภาพจอยบังคับทีละปุ่มโดยมีข้อความแนะนำทีละขั้น เหมาะสำหรับการตรวจรับพัสดุ การตรวจสอบก่อนส่งมอบ และการตรวจหาจุดบกพร่อง ใช้อุปกรณ์เพียงบอร์ด ESP32 หรือ ESP32-S3 และจอยบังคับ โดยไม่ต้องต่ออุปกรณ์อื่น

7.2 ขั้นตอนการทดสอบ

1. เลือกเมนู **File → Examples → Massmore_MomoJoy → 06_FactoryTest** แล้วอัปโหลด
2. เปิด Serial Monitor ที่ความเร็ว **115200 baud**
3. นำจอยเข้าสู่โหมดจับคู่ (สวิตช์ตำแหน่ง **D** และกด **HOME** ค้างประมาณ 3 วินาที) ภายในเวลา 60 วินาที
4. ปฏิบัติตามข้อความที่แสดงทีละขั้น จนครบ 18 ขั้นตอน

โปรแกรมจะ **ล้างข้อมูลการจับคู่ทุกครั้งที่บอร์ดเริ่มทำงาน** เพื่อให้การทดสอบจอยแต่ละตัวเริ่มจากสภาพเดียวกัน

ลำดับ	รายการทดสอบ	ลำดับ	รายการทดสอบ
1	ปุ่ม A	10	ปุ่ม R3 (กดก้านขวา)
2	ปุ่ม B	11	D-Pad ขึ้น
3	ปุ่ม X	12	D-Pad ลง
4	ปุ่ม Y	13	D-Pad ซ้าย
5	ปุ่ม L1	14	D-Pad ขวา
6	ปุ่ม R1	15	ก้านซ้าย หมุนรอบขอบ
7	ไก L2 กดสุด	16	ก้านขวา หมุนรอบขอบ
8	ไก R2 กดสุด	17	ปุ่ม SELECT
9	ปุ่ม L3 (กดก้านซ้าย)	18	ปุ่ม START

เกณฑ์การผ่าน: แต่ละขั้นต้องดำเนินการภายใน 20 วินาที ก้านแต่ละแกนต้องมีระยะเคลื่อนที่อย่างน้อย 600 (จาก 1024) และไกต้องให้ค่าอย่างน้อย 800 (จาก 1023) หากกดปุ่มผิด โปรแกรมจะแจ้งชื่อปุ่มที่กดและรอการกดปุ่มที่ถูกต้องต่อไป

ตัวอย่างข้อความระหว่างการทดสอบ

```
STEP 0/18  pair the joystick
        -> connected
#RESULT PAIR=PASS

STEP 1/18  press  A
        -> PASS

STEP 15/18 rotate the LEFT stick fully around the edge
        X span 1019   Y span 1020
        -> PASS
```

คำสั่งจากแป้นพิมพ์ในหน้าต่าง Serial Monitor

การป้อนคำสั่ง	ผลลัพธ์
พิมพ์ s แล้วกด Enter	ข้ามขั้นตอนปัจจุบัน (บันทึกเป็น SKIP)
พิมพ์ r แล้วกด Enter	ทดสอบซ้ำกับจอยตัวเดิม
กดปุ่ม RST บนบอร์ด	เริ่มทดสอบจอยตัวถัดไป (ล้างข้อมูลการจับคู่ให้อัตโนมัติ)

7.3 รายงานผลการทดสอบ

```
========================================
 TEST REPORT
========================================
 link drop  : no
 BTN_A      PASS
 ...
 BTN_START   PASS
 stick left  X:1019  Y:1020   (need 600)
 trigger L2:1023  R2:1023     (need 800)
 pass 18   fail 0   skip 0   of 18
========================================
#VERDICT PASS
```

บรรทัดที่ขึ้นต้นด้วย #TEST #RESULT และ #VERDICT ออกแบบให้โปรแกรมบนคอมพิวเตอร์อ่านไปประมวลผลต่อได้ ส่วนบรรทัดอื่นเป็นข้อความสำหรับผู้ทดสอบ ผลการทดสอบสามารถบันทึกลงแบบฟอร์มในภาคผนวก ข

> **หมายเหตุ** ปุ่ม HOME ไม่อยู่ในรายการทดสอบ เนื่องจากจอยใช้ปุ่มนี้สำหรับเปิด-ปิดเครื่องและเข้าสู่โหมดจับคู่ ซึ่งจอยจัดการภายในโดยไม่ส่งค่าออกมา หากต้องการทดสอบปุ่มนี้กับจอยที่ส่งค่าได้ ให้กำหนด #define TEST_HOME_BUTTON 1

7.4 เฟิร์มแวร์สำเร็จรูป

โฟลเดอร์ firmware ในคลังข้อมูล GitHub มีไฟล์เฟิร์มแวร์ที่คอมไพล์ไว้แล้วสำหรับบอร์ด ESP32-S3 สามารถติดตั้งได้โดยไม่ต้องคอมไพล์

ไฟล์	ตำแหน่งเขียน (Offset)	รายละเอียด
MomoJoy_FactoryTest_esp32s3_merged.bin	**0x0**	ไฟล์รวม (Bootloader + Partition Table + boot_app0 + โปรแกรม) **ใช้ไฟล์นี้กับเว็บไซต์สำหรับติดตั้งเฟิร์มแวร์**
MomoJoy_FactoryTest_esp32s3.bin	0x10000	เฉพาะส่วนโปรแกรม
bootloader_esp32s3.bin	0x0	Bootloader
partitions_esp32s3.bin	0x8000	Partition Table

การติดตั้งด้วยโปรแกรม esptool

```
pip install -U esptool
esptool.py --chip esp32s3 --port <ชื่อพอร์ต> --baud 921600 \
        write_flash -z 0x0 MomoJoy_FactoryTest_esp32s3_merged.bin
```

8. การแก้ไขปัญหา (Troubleshooting)

8.1 ปัญหาการคอมไพล์และอัปโหลด

อาการ	สาเหตุ	วิธีแก้ไข
ไม่พบพอร์ตในเมนู Tools → Port	ยังไม่ได้ติดตั้งไดรเวอร์ หรือใช้สายสำหรับชาร์จไฟเท่านั้น	ติดตั้งไดรเวอร์ตามบทที่ 3 ขั้นตอนที่ 2 และเปลี่ยนเป็นสาย USB ที่รับส่งข้อมูลได้
'ledcAttach' was not declared in this scope	ติดตั้ง ESP32 Core เวอร์ชัน 2.x	เปลี่ยนเป็นเวอร์ชัน **3.x.x** ใน Boards Manager
Sketch too big (บอร์ด ESP32)	Partition Scheme มีพื้นที่ไม่พอ	เลือก **Huge APP (3MB No OTA/1MB SPIFFS)**
NimBLEDevice.h: No such file or directory	ยังไม่ได้ติดตั้งไลบรารี NimBLE-Arduino	ติดตั้งไฟล์ NimBLE-Arduino.zip ตามบทที่ 3 ขั้นตอนที่ 5
ตัวอย่างในเมนู Examples ไม่ตรงกับคู่มือ	มีไลบรารีเวอร์ชันเก่าค้างอยู่	ลบโฟลเดอร์ MomoJoy และ Massmore_MomoJoy ในโฟลเดอร์ไลบรารี แล้วติดตั้งใหม่
Failed to connect to ESP32	บอร์ดไม่เข้าสู่โหมดดาวน์โหลด	กด **BOOT** ค้าง กด **RST** หนึ่งครั้ง ปล่อย **RST** แล้วปล่อย **BOOT** จากนั้นอัปโหลดใหม่
อัปโหลดหยุดกลางคัน หรือ Timed out	ความเร็วอัปโหลดสูงเกินไป หรือสาย USB คุณภาพต่ำ	ลด Upload Speed เป็น 460800 และเปลี่ยนสาย USB
Permission denied บน Linux	บัญชีผู้ใช้ไม่มีสิทธิ์ใช้พอร์ต	เพิ่มบัญชีเข้ากลุ่ม dialout แล้วออกจากระบบและเข้าใหม่

8.2 ปัญหาการเชื่อมต่อจอย

อาการ	สาเหตุ	วิธีแก้ไข
Serial Monitor ไม่แสดงข้อความ (ESP32-S3)	ตั้งค่า USB CDC On Boot เป็น Enabled	ตั้งค่าเป็น **Disabled** และตรวจสอบความเร็ว 115200 baud
บอร์ดค้นหาจอยไม่พบ	จอยไม่อยู่ในโหมดจับคู่	เลื่อนสวิตช์ไปที่ **D** แล้วกด HOME ค้างประมาณ 3 วินาทีจนไฟกะพริบถี่
เชื่อมต่อแล้วหลุดทันที	ข้อมูลการจับคู่เดิมไม่ตรงกัน	ล้างข้อมูลการจับคู่ด้วย SELECT + START ค้าง 1.5 วินาที (ตัวอย่าง 01) หรือเรียก MomoJoy.forgetBonds() แล้วจับคู่ใหม่
แสดงข้อความ no HID service (0x1812)	จอยอยู่ในโหมดที่ไม่ใช่ BLE HID	เลื่อนสวิตช์โหมดไปที่ตำแหน่ง **D**
แสดงข้อความ encryption failed	ข้อมูลการจับคู่ไม่ตรงกัน	ล้างข้อมูลการจับคู่แล้วจับคู่ใหม่
สัญญาณหลุดบ่อยระหว่างใช้งาน	สัญญาณรบกวนย่าน 2.4 GHz	ย้ายบริเวณใช้งานให้ห่างจากอุปกรณ์กระจายสัญญาณ Wi-Fi และพอร์ต USB 3.0
บอร์ดเชื่อมต่อกับจอยผิดตัว	มีจอยหลายตัวเปิดอยู่ในบริเวณเดียวกัน	ระบุจอยด้วย MAC Address ตามหัวข้อ 4.3

8.3 ปัญหาระหว่างการใช้งานหุ่นยนต์

อาการ	สาเหตุ	วิธีแก้ไข
มอเตอร์หมุนเองเมื่อเปิดเครื่อง	ขาควบคุมวงจรขับมอเตอร์อยู่ในสถานะลอย	เรียก parkMotorPins() เป็นคำสั่งแรกใน setup()
ต่อวงจรถูกต้องแต่มอเตอร์ไม่หมุน (SKU-1020)	ไม่ได้เปิดสวิตช์ Power บนบอร์ด	เปิดสวิตช์ Power เพื่อใช้ไฟจากแบตเตอรี่
มอเตอร์ไม่หมุน (SKU-1015)	ไม่ได้ติดตั้งโมดูลขับมอเตอร์	ติดตั้งโมดูลขับมอเตอร์ตามหัวข้อ 5.3
ก้านเคลื่อนเองทั้งที่ไม่ได้สัมผัส	ก้านอนาล็อกคลาดเคลื่อน (Stick Drift)	เพิ่ม Dead Zone เช่น MomoJoy.setDeadzone(48)
หุ่นยนต์ออกตัวแรงเกินไป	ค่าความเร็วสูงเกินไป	ลด SPEED_NORMAL และลด SLEW_UP เป็น 4
หุ่นยนต์สั่นแต่ไม่เคลื่อนที่	ค่า PWM ต่ำกว่าที่มอเตอร์เริ่มหมุน	เพิ่ม MOTOR_MIN_DUTY เป็น 50–60
ดันก้านไปข้างหน้าแต่หุ่นยนต์ถอยหลัง	ไม่ได้กลับเครื่องหมายค่า ly() หรือสายมอเตอร์สลับขั้ว	ใช้ -MomoJoy.ly() หรือสลับสายมอเตอร์

8.4 การปรับเทียบปุ่มสำหรับจอยรุ่นอื่น

หากนำไลบรารีไปใช้กับจอย BLE HID รุ่นอื่น แล้วชื่อปุ่มที่แสดงไม่ตรงกับปุ่มที่กด ให้ดำเนินการดังนี้

1. เปิดตัวอย่าง 02_ReadAllSerial แล้วกำหนด #define SHOW_RAW_REPORTS 1
2. อัปโหลดแล้วเปิด Serial Monitor จะปรากฏตาราง HID Report Map และข้อมูลดิบของทุก Report
3. กดปุ่มทีละปุ่ม สังเกตบิตที่เปลี่ยนแปลง แล้วบันทึกหมายเลขปุ่ม HID
4. สร้างตารางปุ่มของผู้ใช้งาน แล้วกำหนดให้กับ opt.profile

```
static const MomoButtonMap kMyMap[] = {
   {1, MOMO_BTN_A}, {2, MOMO_BTN_B}, {4, MOMO_BTN_X}, {5, MOMO_BTN_Y},
   // ... เพิ่มหมายเลขปุ่มตามที่ตัวอย่าง 02 แสดง
};
static const MomoProfile kMyProfile = {"my pad", kMyMap,
   (uint8_t)(sizeof(kMyMap) / sizeof(kMyMap[0])), false, false};

MomoJoyOptions opt;
opt.profile = &kMyProfile;
MomoJoy.begin(opt);
```

9. ข้อแนะนำสำหรับการใช้งาน (Pro Tips)

1. **ระบบหยุดฉุกเฉินต้องมีเสมอ** ในหุ่นยนต์ที่มีล้อ ไม่ควรตรวจสอบเพียง isConnected() แต่ควรตรวจสอบ msSinceReport() > 400 ร่วมด้วย เนื่องจากอาจมีกรณีที่สถานะยังเชื่อมต่อแต่จอยหยุดส่งข้อมูลแล้ว
2. **ปรับ Dead Zone ให้เหมาะกับงาน** งานเกมใช้ค่าประมาณ 16–24 ส่วนงานหุ่นยนต์ควรใช้ 40–48 เพื่อป้องกันหุ่นยนต์เคลื่อนที่เองเมื่อปล่อยก้าน
3. **ใช้ไกอนาล็อกเป็นคันเร่ง** คำสั่ง l2f() / r2f() ให้ค่า 0.0–1.0 นำไปคูณกับความเร็วสูงสุดได้โดยตรง ให้การควบคุมที่นุ่มนวลกว่าการใช้ปุ่มกด
4. **ตรวจสอบคุณภาพสัญญาณก่อนใช้งานจริง** ค่า reportRateHz() ปกติอยู่ที่ประมาณ 60–125 Hz หากต่ำกว่า 30 Hz แสดงว่าสัญญาณถูกรบกวนหรือระยะไกลเกินไป
5. **แจ้งเตือนแบตเตอรี่ต่ำ** ใช้ค่า battery() ร่วมกับ Buzzer หรือจอแสดงผล เพื่อแจ้งเตือนเมื่อแบตเตอรี่ของจอยต่ำกว่า 20%
6. **ระบุจอยด้วย MAC Address ในงานที่ต้องการความแน่นอน** เช่น ห้องเรียนและสายการผลิต เพื่อป้องกันการเชื่อมต่อสลับเครื่อง
7. **จัดเตรียมบอร์ดสำหรับตรวจสอบจอย** ติดตั้งเฟิร์มแวร์ Factory Test ไว้บนบอร์ดสำรองหนึ่งบอร์ด เพื่อใช้ตรวจสอบจอยแต่ละตัวได้ภายในเวลาไม่เกิน 2 นาที

10. แหล่งข้อมูลอ้างอิง (Resources)

10.1 คลังข้อมูล GitHub

รายการ	เนื้อหา	ที่อยู่
MOMO Joystick (SKU-1001)	ไลบรารี Massmore_MomoJoy โปรแกรมตัวอย่าง 6 ชุด และเฟิร์มแวร์	https://github.com/Massmore/Massmore_MOMOJoystick_SKU-1001
MOMO ESP32-S3 (SKU-1020)	คู่มือและตัวอย่างของบอร์ด MOMO ESP32-S3	https://github.com/Massmore/SKU-1020_MOMO
ESP32 Breakout (SKU-1015)	คู่มือและตัวอย่างของบอร์ด ESP32 Breakout	https://github.com/Massmore/Massmore_ESP32Breakout_SKU-1015
NimBLE-Arduino	ไลบรารี BLE ที่ใช้ในชุดนี้	https://github.com/h2zero/NimBLE-Arduino
arduino-esp32	ESP32 Arduino Core ของ Espressif	https://github.com/espressif/arduino-esp32
pioarduino platform-espressif32	แพลตฟอร์ม PlatformIO ที่รองรับ Core 3.x	https://github.com/pioarduino/platform-espressif32

10.2 ผลิตภัณฑ์ที่เกี่ยวข้อง

ผลิตภัณฑ์	ที่อยู่
บอร์ดพัฒนาหุ่นยนต์ MOMO ESP32-S3 (SKU-1020)	https://www.massmore.shop/products/743deb7c-556d-41b7-b42a-1e9d6df55bca
บอร์ดขยาย ESP32 38PIN Breakout (SKU-1015)	https://www.massmore.shop/products/1cbd7c12-166a-4cbb-bbc6-3d7fdce0775c
ร้านค้า Massmore	https://www.massmore.shop

10.3 มาตรฐานที่อ้างอิง

เอกสาร	เนื้อหาที่อ้างอิง	ที่อยู่
HID over GATT Profile (HOGP)	โปรไฟล์ BLE สำหรับอุปกรณ์ HID	https://www.bluetooth.com/specifications/specs/hid-over-gatt-profile-1-0/
HID Service (HIDS)	Characteristic 0x2A4B 0x2A4D 0x2A4E	https://www.bluetooth.com/specifications/specs/hid-service-1-0/
USB HID Usage Tables	ความหมายของ Usage Page และ Usage	https://www.usb.org/document-library/hid-usage-tables-16
Bluetooth Assigned Numbers	หมายเลข Service 0x1812, 0x180F และ Appearance 0x03C4	https://www.bluetooth.com/specifications/assigned-numbers/

10.4 เอกสารข้อมูลวงจรรวม (Datasheet)

วงจรรวม	หน้าที่	ใช้บนบอร์ด	ที่อยู่
ESP32-S3-WROOM-1 / 1U	ไมโครคอนโทรลเลอร์ Wi-Fi และ BLE	SKU-1020	https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
ESP32-WROOM-32	ไมโครคอนโทรลเลอร์ Wi-Fi และ BLE	SKU-1015	https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf
CH343P	วงจรแปลง USB เป็น Serial	SKU-1020	https://www.wch-ic.com/products/CH343.html
TB67H451AFNG	วงจรขับมอเตอร์ DC	SKU-1020	https://toshiba.semicon-storage.com/ap-en/semiconductor/product/motor-driver-ics/brushed-dc-motor-driver-ics/detail.TB67H451AFNG.html
PCA9685PW	วงจรสร้างสัญญาณ PWM 16 ช่องผ่าน I2C	SKU-1020	https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf
TPS62162	วงจรแปลงแรงดันแบบสวิตชิ่ง 3.3 V	SKU-1020	https://www.ti.com/product/TPS62162
TB6612FNG	วงจรขับมอเตอร์ DC (โมดูลภายนอก)	ใช้ร่วมกับ SKU-1015	https://toshiba.semicon-storage.com/ap-en/semiconductor/product/motor-driver-ics/brushed-dc-motor-driver-ics/detail.TB6612FNG.html

10.5 เว็บไซต์อ้างอิง

หัวข้อ	ที่อยู่
Arduino IDE (ดาวน์โหลด)	https://www.arduino.cc/en/software
ESP32 Arduino Core Documentation	https://docs.espressif.com/projects/arduino-esp32/en/latest/
ESP32-S3 Technical Reference Manual	https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
PlatformIO Documentation	https://docs.platformio.org/

10.6 สำหรับผู้ใช้งาน PlatformIO

ผู้ใช้งาน Visual Studio Code สามารถเปิดโฟลเดอร์ **PlatformIO** จากคลังข้อมูลได้ทันที โดยไม่ต้องติดตั้งไลบรารีเพิ่ม เนื่องจากไลบรารี NimBLE-Arduino อยู่ในโฟลเดอร์ lib แล้ว การเลือกโปรแกรมตัวอย่างทำได้โดยแก้ไขค่าในไฟล์ src/main.cpp

```
#define MOMO_EXAMPLE 1   // เลือกหมายเลขตัวอย่าง 1-6
```

Environment	บอร์ดที่กำหนด	การใช้งาน
esp32s3	esp32-s3-devkitc-1	บอร์ด MOMO ESP32-S3 (SKU-1020)
esp32	esp32dev	บอร์ด ESP32 Breakout (SKU-1015)
factorytest_esp32s3	esp32-s3-devkitc-1	สร้างเฟิร์มแวร์ Factory Test สำหรับ ESP32-S3
factorytest_esp32	esp32dev	สร้างเฟิร์มแวร์ Factory Test สำหรับ ESP32

ภาคผนวก ก สรุปคำสั่งของไลบรารี

```
#include <Massmore_MomoJoy.h>
using namespace massmore;

void setup() {
  Serial.begin(115200);
  MomoJoy.begin();              // ค้นหา จับคู่ และเชื่อมต่ออัตโนมัติ
}

void loop() {
  MomoJoy.update();             // ต้องเรียกทุกรอบของ loop()
  if (!MomoJoy.isConnected()) return;

  if (MomoJoy.justPressed(MOMO_BTN_A)) Serial.println("A");

  int speed = -MomoJoy.ly();    // -512..511 (ดันขึ้นได้ค่าติดลบ จึงกลับเครื่องหมาย)
  int brake =  MomoJoy.l2();    //    0..1023
}
```

กลุ่มคำสั่ง	คำสั่ง
การเริ่มต้นและสถานะ	begin(opt) update() isConnected() isScanning() rescan() disconnect() forgetBonds() end()
การเลือกจอย	scanDevices(ms) foundCount() foundDevice(i) setAddressFilter() setNameFilter() connectToAddress()
ปุ่มกด	pressed() justPressed() justReleased() longPressed() heldMs() anyButton() buttons()
แกนอนาล็อก	lx() ly() rx() ry() l2() r2() · lxf() ถึง r2f() · mapStick(v, max)
D-Pad	dpad() dpadUp() dpadDown() dpadLeft() dpadRight() dpadJustChanged()
ข้อมูลการเชื่อมต่อ	battery() rssi() peerName() peerAddress() packets() reportRateHz() msSinceReport() version()
การปรับแต่ง	setDeadzone() setTriggerThreshold() setProfile() setInvertLeftY() setInvertRightY()
ฟังก์ชันตอบสนองเหตุการณ์	onConnect() onDisconnect() onButton() onDpad() onRawReport()
การแสดงผลและดีบัก	printState() printStateChanges() printCsv() printJson() dumpReportMap() dumpDescriptorFields()

ค่าตั้งต้นใน MomoJoyOptions

ตัวแปร	ค่าเริ่มต้น	ความหมาย
localName	"MOMO"	ชื่อ BLE ของบอร์ด
nameFilter	nullptr	เชื่อมต่อเฉพาะจอยที่ชื่อมีข้อความนี้
addressFilter	nullptr	เชื่อมต่อเฉพาะจอยที่มี MAC Address นี้
autoReconnect	true	ค้นหาใหม่อัตโนมัติเมื่อการเชื่อมต่อหลุด
clearBondsOnBoot	false	ล้างข้อมูลการจับคู่ทุกครั้งที่เริ่มทำงาน
deadzone	24	Dead Zone แบบรัศมี (0–512)
connectTimeoutMs	10000	เวลาสูงสุดในการเชื่อมต่อแต่ละครั้ง (มิลลิวินาที)

ตัวแปร	ค่าเริ่มต้น	ความหมาย
triggerThreshold	64	ค่า L2 / R2 ที่ถือว่ากดไก
txPowerDbm	9	กำลังส่ง BLE (−12 ถึง +9 dBm)
autoStart	true	เริ่มค้นหาจอยทันทีเมื่อเรียก begin()
verbose	true	แสดงขั้นตอนการทำงานทาง Serial

ภาคผนวก ข แบบบันทึกผลการทดสอบจอยบังคับ

รายการ	ข้อมูล
วันที่ทดสอบ	
ผู้ทดสอบ	
หมายเลขจอย / MAC Address	
บอร์ดที่ใช้ทดสอบ	□ ESP32 Breakout (SKU-1015) □ MOMO ESP32-S3 (SKU-1020)
ระดับแบตเตอรี่ (%)	

ลำดับ	รายการทดสอบ	ผ่าน	ไม่ผ่าน	หมายเหตุ
0	การจับคู่และเชื่อมต่อ	□	□	
1–4	ปุ่ม A, B, X, Y	□	□	
5–6	ปุ่ม L1, R1	□	□	
7–8	ไก L2, R2 (ค่าไม่น้อยกว่า 800)	□	□	
9–10	ปุ่ม L3, R3	□	□	
11–14	D-Pad ขึ้น ลง ซ้าย ขวา	□	□	
15–16	ก้านซ้าย ก้านขวา (ระยะไม่น้อยกว่า 600)	□	□	
17–18	ปุ่ม SELECT, START	□	□	
—	ผลสรุป (#VERDICT)	□ PASS	□ FAIL	

ลงชื่อ ..... ผู้ทดสอบ ลงชื่อ ..... ผู้ตรวจสอบ